密级：秘密

文档编号：D0000-PPC-RA2026-PPD-2026

项目名称：RentAgent —— 基于 AI 智能体的房屋租赁系统

项目编号：RA2026

概要设计

版本：V1.8

2026-09-20

大连理工大学创新实践基地

（版权所有，翻版必究）

总页数	59	正文	55	附录	无	生效日期	2026. 09. 20
编制	王硕			批准	姜厚云		

文件修改控制

修改编号	版本	修改条款及内容	修改日期
1	V1.0	创建。依《概要设计模板》编写 RentAgent 概要设计：系统结构图、七大模块的功能定义 / 模块结构 / 类图与接口说明、数据库引擎与表间关系图、全局变量与模块间接口函数设计、关键数据结构定义	2026-09-16
2	V1.1	① 按《项目最终确认书》v1.4 与《概要设计说明》v1.4 同步客户端交付形态：系统结构图（图 2-1）与第 2 章表现层叙述补充“浏览器 Web 版 + Electron 桌面端”两种分发形态——两版共用同一份构建产物，桌面端外壳仅注入后端地址、不新增界面与功能；② 按《项目最终确认书》v1.5 与《概要设计说明》v1.5 明确大模型口径为模型无关：图 2-1 与 AI 智能体模块类图中的模型服务节点改为“任意兼容模型”表述，不再把系统与某一厂商绑定	2026-09-17
3	V1.2	新增 AI 对话审计与工具调用留痕设计，并按实现同步：① 第 3 章 AI 智能体服务模块新增“对话审计与工具留痕”功能项与 HousingToolProvider / ToolTraceService 两个类（图 3-8 同步），后台管理模块新增“AI 对话审计”功能项与管理端接口函数（图 3-14 同步）；AI 与后台管理的模块结构图同步补入工具提供器、工具留痕服务与对话审计服务；② 第 4 章关键数据结构补 AdminChatDto（会话列表 / 会话详情 / 消息 / 规模统计），并修正业务状态字典中 ai_chat_message.role 的取值（1 用户 / 2 助手 / 3 工具，原记为 0/1/2）；③ 第 5 章接口函数清单补充 GET /admin/chats 与 GET /admin/chats/{id}，并在 5.2 节补两份对应接口规约（chats / chatDetail）	2026-09-17
4	V1.3	按《概要设计说明》v1.7“移除全部模拟数据、AI 全部调用真实模型”口径重构 AI 模块：① 删除规则引擎实现 MockAgent 与“无 Key 自动降级”链路，未配置模型或调用失败统一返回 4001；② 新增结构化调用层 AiJsonClient 与提示词库 AiPrompts，定价 / 虚假检测 / 合同解读 / 识别填充改为“数据事实 → 模型 JSON → 校验 → 落 ai_analysis”；③ 客服来源引用	2026-09-17

修改编号	版本	修改条款及内容	修改日期
		改由 searchKnowledge 工具返回体捕获（NFR-05）；④ 新增流式工具调用兜底（丢失 tool_calls 时非流式重跑一次）；图 2-1、图 3-7、图 3-8 同步更新	
5	V1.4	按《概要设计说明》v1.8 修正流式工具调用链路并强化结构化输出：① openai-chat-completions 协议改由本项目自研适配器 OpenAiChatCompletionsModel 实现（三种协议全部自研，不再使用 langchain4j-open-ai），同一轮内的正文与 tool_calls 分别累积后合并为一个 AiMessage——LangChain4j 0.35 的流式组装器在正文非空时会丢弃工具调用，是客服场景“只回一句过程语、工具从未执行”的真正根因；② 四项分析调用改以 response_format=json_schema（strict）强约束输出结构，Schema 由目标类型经 JsonSchemas 生成，端点不支持时自动退回提示词约束；③ 非流式兜底仅在本轮未执行任何工具时才启用（工具已执行的短回答不再重跑）；图 3-7、图 3-8 同步更新	2026-09-17
6	V1.5	按《概要设计说明》v1.9 与《数据库设计简介》v1.5 校正实现口径（需求基线不变）：① 预约表增 active_slot（时段占位键）列，唯一键由 (house_id, appointment_time) 改为 UNIQUE(active_slot)——仅待确认/已确认持有占位键，终态置 NULL 释放时段，使已取消/已拒绝的时段可被重新预约；预约时间须晚于当前时间；② 合同详情（detailContract）限合同双方与管理员可读，越权返回 1007；③ 房源图片写入同步刷新 house.cover_url 冗余列（列表卡片只读该列），编辑请求未携带 images 字段视为“本次不改动图片”；④ 合同解读回写 risk_flags 改为只更新该列，避免整行回写覆盖并发的合同状态变更；⑤ 助手消息落库以终止事件正文兜底（部分协议流式 delta 全程为空）；第 3 章交易与合同模块、第 4 章表 4-1/表 4-3、第 5 章对应接口规约同步更新	2026-09-18
7	V1.6	按前后端协同代码评审结论统一契约与实现口径（需求基线 FR-01~25 / NFR-01~10 不变）：① 同一逻辑字段只保留一种形状——GET /houses 搜索结果由 Item{house,images,landlordName…}	2026-09-18

修改编号	版本	修改条款及内容	修改日期
		改为裸房源分页 R<PageVO<House>>（列表卡片只读实体字段，原形状每行白做 4 次查询，前端还需按形状嗅探）；house.facilities（JSON 列）由实体侧 typeHandler 统一映射为 List<String>，消除“入参数组 / 出参字符串 / AI 填充数组”三种形状与随之而来的前后端兼容分支；② 列表响应全面 record 化：预约 / 评价 / 合同 / 订单 / 举报 / 账号 / 看板 / 当前用户等原以 Map<String,Object> 返回的 VO 改为 record（OpenAPI 出 schema，前端类型可对齐），合同列表直接携带 houseTitle，前端不再逐份合同回查 /houses/{id}；③ JSON 列读写收敛到 JsonColumns，替换原先在四处各写一份的序列化与容错解析；④ 控制器不再自造响应：房源审核 / 举报处理入参改 DTO + 参数校验（缺 pass 字段不再被静默当成“驳回”），通知与留痕下沉到 Service，错误码统一走 ErrorCode（新增 NOT_LOGIN / WRONG_PASSWORD / REPORT_NOT_FOUND 复用码说明）；⑤ AI 对话链路修四处：推理内容占满 max_tokens 时正文与工具调用可能全空，适配器原会伪造一句“没有生成有效回答”且不发 delta（界面空白气泡、CI 三条断言莫名其妙失败）——现改为如实报错并按可自愈传输层故障（空返回 / 限流 / 5xx / 超时）非流式重跑一次，4xx 不重跑；流式分支增加 HTTP 状态码校验与异常解包；正文只在终止消息里给出时补走 delta 通道；单轮 token 上限由 2048 提到 4096（实测一轮约 1.6k~2.0k tokens，余量不足 20% 即被 finish_reason=length 截断）；⑥ 后台审计轨迹改用不抛错的引擎描述，未配置模型的实例也能查看留痕；新增“操作留痕”后台页面；新增房源图片上传入口，补上 FR-05 的图片链路；地图找房改用 GET /houses/map（不再只画当前页 10 个点）；⑦ 同步第 3/4/5 章与第 8/9 章、以及《单元测试用例设计》v1.3（后端 190 条）、《测试用例设计》v1.3（集成断言 130 条）	
8	V1.7	找房智能体工具能力升级（需求基线不变）：① searchHouses 工	2026-09-19

修改编号	版本	修改条款及内容	修改日期
		具扩参——新增 minRent（预算下限）、facilities（逗号分隔设施标签）、sort（rent_asc / rent_desc），单次返回 5 → 8 套，返回体改为 {total, list[]}（模型可知房源总数），房源卡片补 coverUrl 与详情页链接 detailUrl（由 HousingTools 统一拼装 #/app/houses/{id}）；② getHouseDetail 工具实装（按 id 查单套房源完整信息，供模型回答追问）；③ 系统提示词约定房源链接协议：推荐逐套输出 [标题](detailUrl) Markdown 链接且原样引用工具返回值，不得编造房源与链接，空结果如实说明；④ 前端聊天气泡把回答中的站内链接渲染为可点击 <a>（历史会话回放天然携带）。SSE 契约与数据结构零变化；图 3-8 类图与 AI 模块结构图的 HousingTools 节点同步	
9	V1.8	全篇排版重构（设计内容与需求基线不变）：① 取消章节与模块的强制分页，按《概要设计参考示例》体例连续流排（该示例第 3 章即紧接图 2-1 同页），消除“整页只有一张图或一张表”的空松页；② 修正表格列宽写入缺陷——固定布局下 Word/LibreOffice 以 tblGrid 为准，此前只写单元格宽度 tcW 而被均分的 tblGrid 覆盖，功能定义表“功能点详细说明”列实际仅 4.25cm（设计值 12.39cm），长文折行致表高翻倍，现同步写 tblGrid 与 tblW；③ 接口说明表移至模块结构图之后、类图之前（结构 → 对外接口 → 内部类），图号表号不变；④ 竖长结构图按限高定宽（13.8 → 11.5cm 高，评价与消息 9.9 → 7.6cm 高），使结构图与接口表合占一页；⑤ 序号 / 类型等窄列加宽，消除表头汉字竖排折行；⑥ 图题 / 表题 / 代码块间距收紧，5.2 节表间空段改紧凑分隔段；⑦ 文件修改控制表拉到正文同宽 16.99cm（说明列 6.94 → 9.5cm）。全篇 74 页 → 59 页，正文页平均底部留白 4.0cm → 1.9cm	2026-09-20

目 录

1 文档概述.....8
1.1 文档目的和范围.....8
1.2 术语/缩略语.....8
1.3 参考文档.....9
2 系统结构图.....9
3 模块详细概述.....11
3.1 用户与权限模块.....11
3.1.1 用户与权限模块功能定义.....12
3.1.2 用户与权限模块结构.....13
3.1.3 用户与权限模块类图.....14
3.2 房源管理模块.....15
3.2.1 房源管理模块功能定义.....15
3.2.2 房源管理模块结构.....16
3.2.3 房源管理模块类图.....17
3.3 检索与推荐模块.....18
3.3.1 检索与推荐模块功能定义.....18
3.3.2 检索与推荐模块结构.....19
3.3.3 检索与推荐模块类图.....20
3.4 AI 智能体服务模块.....21
3.4.1 AI 智能体服务模块功能定义.....21
3.4.2 AI 智能体服务模块结构.....22
3.4.3 AI 智能体服务模块类图.....24
3.5 交易与合同模块.....25
3.5.1 交易与合同模块功能定义.....25
3.5.2 交易与合同模块结构.....26
3.5.3 交易与合同模块类图.....27
3.6 评价与消息模块.....27
3.6.1 评价与消息模块功能定义.....28

3.6.2 评价与消息模块结构.....28
3.6.3 评价与消息模块类图.....29
3.7 后台管理模块.....30
3.7.1 后台管理模块功能定义.....30
3.7.2 后台管理模块结构.....31
3.7.3 后台管理模块类图.....32
4 数据库设计.....33
4.1 数据库引擎概述.....33
4.2 数据库概要设计.....34
5 接口设计.....37
5.1 全局变量.....37
5.2 模块间接口函数设计.....39
5.2.1 用户与权限模块.....39
5.2.2 房源管理模块.....41
5.2.3 检索与推荐模块.....44
5.2.4 AI 智能体服务模块.....46
5.2.5 交易与合同模块.....49
5.2.6 评价与消息模块.....51
5.2.7 后台管理模块.....53
5.3 关键数据结构定义.....56
5.3.1 统一响应结构.....57
5.3.2 分页结构.....57
5.3.3 核心 DTO 结构.....57
5.3.4 核心 VO 结构.....59

1 文档概述

1.1 文档目的和范围

本文档是 RentAgent（基于 AI 智能体的房屋租赁系统）的概要设计说明书，用于在《03-需求分析矩阵》确立的需求基线（功能需求 FR-01~FR-25、非功能需求 NFR-01~NFR-10）之上，给出系统的总体结构、模块划分与模块内结构、数据库设计、模块间接口函数与关键数据结构，作为后续详细设计、编码实现、集成测试与维护的共同基准。

文档范围覆盖 RentAgent 的全部已确认功能模块与支撑性设计：系统分层与部署形态、七大业务模块的职责边界与内部结构、18 张业务表及其联系、对外接口规范与函数级约定、贯穿全系统的状态字典与错误码。需求实现程度的逐条追溯见《08-需求跟踪矩阵》，数据库字段级细节见《数据库设计简介》。

本文档的阅读对象为：项目负责人、系统架构与后端开发人员、前端开发人员、测试人员、数据库设计人员以及课程指导教师与评审组。

1.2 术语/缩略语

本文档使用的术语与缩略语如下表所示。

表 1-1 术语与缩略语

序号	术语/缩略语	说明
1	API	应用程序编程接口（Application Programming Interface）
2	B/S 架构	浏览器 / 服务器架构，本项目三端均以浏览器为运行载体
3	JWT	JSON Web Token，无状态登录凭证，本项目有效期 72 小时
4	RBAC	基于角色的访问控制：租客（1）/ 房东（2）/ 管理员（3）三角色
5	BCrypt	自适应密码哈希算法，用于账号密码存储
6	AES	对称加密算法，用于身份证号等敏感字段的加密存储
7	SSE	Server-Sent Events，服务端推送事件，用于 AI 对话流式输出
8	LLM	Large Language Model，大语言模型
9	RAG	检索增强生成：先检索知识库片段再交由大模型生成可溯源的回答
10	Function Calling	大模型按预定义工具规范发起结构化调用，由系统执行后回传结果
11	Agent 编排	围绕用户目标组织“理解 → 规划 → 调用工具 → 反思 → 交付”的多步闭环

序号	术语/缩略语	说明
12	LLM 网关	对大模型 API 的统一封装层，支持协议切换、超时与降级
13	MVP	最小可行产品，本项目即 P0 需求集合（FR-01/02/05/06/07/09/12/13/17/18）
14	状态机	以受限状态迁移表达业务生命周期，状态更新一律带前态条件（乐观并发控制）
15	逻辑外键	不建物理外键约束、由业务层保证一致性的关联列，兼顾完整性与写入性能
16	DTO / VO	数据传输对象 / 视图对象，分别用于入参装载与出参组装

1.3 参考文档

本文档编写所依据或引用的文档如下表所示。

表 1-2 参考文档

序号	文档名	作者	时间	版本
1	01-项目技术背景介绍	项目组	2026-09-03	v1.4
2	02-项目最终确认书	项目组	2026-09-03	v1.5
3	03-需求分析矩阵	项目组	2026-09-04	v1.1
4	05-需求分析评审报告	姜厚云 / 项目组	2026-09-07	v1.0
5	06-原型设计评审报告	姜厚云 / 项目组	2026-09-07	v1.0
6	07-项目企划书	项目组	2026-09-13	v1.2
7	08-需求跟踪矩阵	项目组	2026-09-07	v1.2
8	概要设计说明	王硕	2026-09-19	v1.11
9	数据库设计简介	李天泽	2026-09-18	v1.6
10	docker/mysql-init/01_schema.sql（建库脚本）	李天泽	2026-09-14	v1.0
11	概要设计模板	大连理工大学创新实践基地	2026-01-01	0.8.0-0.0.0

2 系统结构图

RentAgent 采用 B/S 架构与前后端分离形态，自顶向下划分为表现层、接入层、接口层、业务模块层、技术分层与数据层，属于典型的应用项目架构。

表现层为单仓单应用的三端前端（React 18 + TypeScript），按登录角色在哈希路由上分区：租客端负责找房、AI 对话、预约与签约；房东端负责房源发布、定价分析、预约处理与订单管理；管理端负责

房源审核、用户管理、举报处理与数据看板。三端复用同一套组件与请求封装，差异主要体现在路由与权限。客户端有两种分发形态，共用同一份构建产物：浏览器 Web 版由 Nginx 托管静态资源（构建基址为相对路径，可直接部署在任意子路径），桌面端由 Electron 外壳加载该产物（frontend/electron/），外壳只负责开窗并注入后端服务地址（file:// 下相对路径无法请求后端），不新增任何界面与功能；后端接口基址在页面侧统一由 src/runtime.ts 解析，两种形态因此使用同一套接口调用代码。

接入层由 Nginx 承担静态资源托管、HTTPS 终止、/api 到后端 server:8080 的反向代理与 /uploads 图片卷的直出；接口层统一以 /api/v1 为前缀对外提供 RESTful 接口，并以 R<T> 统一响应体、JWT 鉴权过滤器与角色拦截器、全局异常处理器作为横切关注点。

业务模块层为七大模块（详见第 3 章）：用户与权限、房源管理、检索与推荐、交易与合同、评价与消息、后台管理，以及作为项目特色的 AI 智能体服务模块。技术分层上，各业务模块统一遵循 Controller → Service → Mapper 的调用链，AI 智能体服务模块额外在 Service 之下引入 Agent 编排层与 LLM 网关。

数据层由 MySQL 8（业务主库）、Redis 7（验证码、登录失败计数）、Redis Stack（RAG 向量索引升级位）与本地图片卷组成；大模型服务作为外部依赖经 HTTPS 出网调用——AI 能力全部依赖真实模型，未配置模型或调用失败时统一返回 4001（不做规则/模拟兜底）。

系统结构如图 2-1 所示。

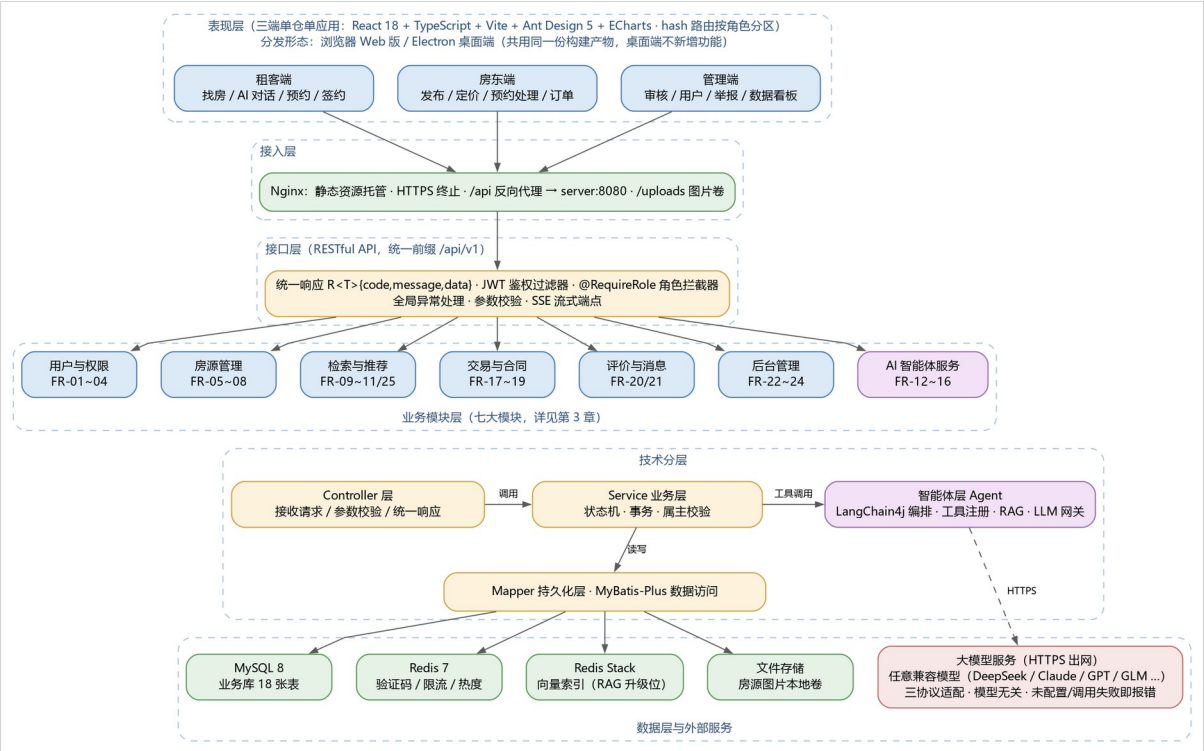


图 2-1 系统结构图

3 模块详细概述

RentAgent 的三端均为浏览器应用，业务能力按职责聚合为七个模块，与《03-需求分析矩阵》的功能模块划分一致。模块间通过 Service 接口调用与数据库共享协作，禁止跨模块直接访问对方 Mapper，以保证模块边界的可分性与可测试性。

需要说明架构关注点的差异：课堂讲授的嵌入式项目以“感知层—控制层—执行层”为主线，强调实时性与资源受限；本项目为 B/S 应用项目，分层主线为“表现层 → 业务层 → 智能体层 → 数据层”，关键指标为并发能力、响应时间与可维护性。本项目不存在设备侧感知与实时控制环节，故不引入嵌入式分层。

表 3-1 模块划分总览

序号	模块名称	覆盖需求	主要职责	主要实现组件
1	用户与权限模块	FR-01 ~ FR-04	注册登录、实名认证、个人信息、登录态与角色鉴权	AuthService · JWT 过滤器 · 角色拦截器
2	房源管理模块	FR-05 ~ FR-08	房源发布、编辑上下架、管理员审核、图片与智能识别	HouseService · AdminController（审核）
3	检索与推荐模块	FR-09 ~ FR-11、FR-25	关键词与多条件检索、地图找房、推荐、收藏	SearchService · FavoriteController
4	AI 智能体服务模块	FR-12 ~ FR-16	找房助手、智能客服、合同解读、定价、虚假检测	ChatService · KbService · AnalysisService · LlmGateway
5	交易与合同模块	FR-17 ~ FR-19	预约看房、在线签约、租赁订单与租金账单	AppointmentService · ContractService
6	评价与消息模块	FR-20、FR-21	评价提交与展示、站内通知、举报受理入口	ReviewService · NotificationService
7	后台管理模块	FR-22 ~ FR-24	用户管理、审核与举报处理、审计留痕、统计看板	AdminService · ReportService · AuditLogService

3.1 用户与权限模块

用户与权限模块承担系统的入口能力：账号注册与登录、实名认证、个人信息维护，以及贯穿所有模块的登录态校验与角色鉴权。模块采用无状态 JWT 方案，令牌不落库，服务端仅校验签名与有效期；账号禁用通过每次请求校验用户状态即时生效，避免维护令牌黑名单。

密码以 BCrypt 哈希存储；身份证号等敏感字段以 AES 加密落库并另存 SHA-256 哈希用于比对，满足 NFR-04 的隐私合规要求。

3.1.1 用户与权限模块功能定义

表 3-2 用户与权限模块功能定义

序号	功能点	功能点详细说明
1	用户注册	手机号 + 验证码 + 密码注册，注册时即选择角色（1 租客 / 2 房东）；手机号唯一性校验；注册成功自动登录并跳转对应角色首页（FR-01）
2	验证码下发	向手机号下发验证码并写入 Redis（5 分钟 TTL）；演示期使用固定码 246810，生产接入短信服务商后置空（FR-01）
3	用户登录	支持用户名或手机号 + 密码登录，BCrypt 比对；连续错误 5 次锁定该账号 10 分钟；成功后签发有效期 72 小时的 JWT（FR-02）
4	实名认证	房东提交真实姓名与身份证号；证件号 AES 加密存储 + SHA-256 哈希比对字段；认证状态为待审核 / 已通过 / 已驳回，一人可多次提交，业务层取最新记录生效（FR-03）
5	个人信息管理	查询与修改昵称、邮箱、头像；修改密码须校验原密码（FR-04）
6	登录态与状态校验	JWT 无状态鉴权；每次请求校验账号状态，管理员禁用后立即无法访问；令牌有效期内的登出由前端清除凭证实现（FR-02 / FR-22）
7	角色鉴权	三角色 RBAC：方法级 @RequireRole 注解 + 角色拦截器统一拦截；资源操作叠加属主校验，防止水平越权与垂直越权（NFR-03）

3.1.2 用户与权限模块结构

用户与权限模块的模块结构如下图所示。

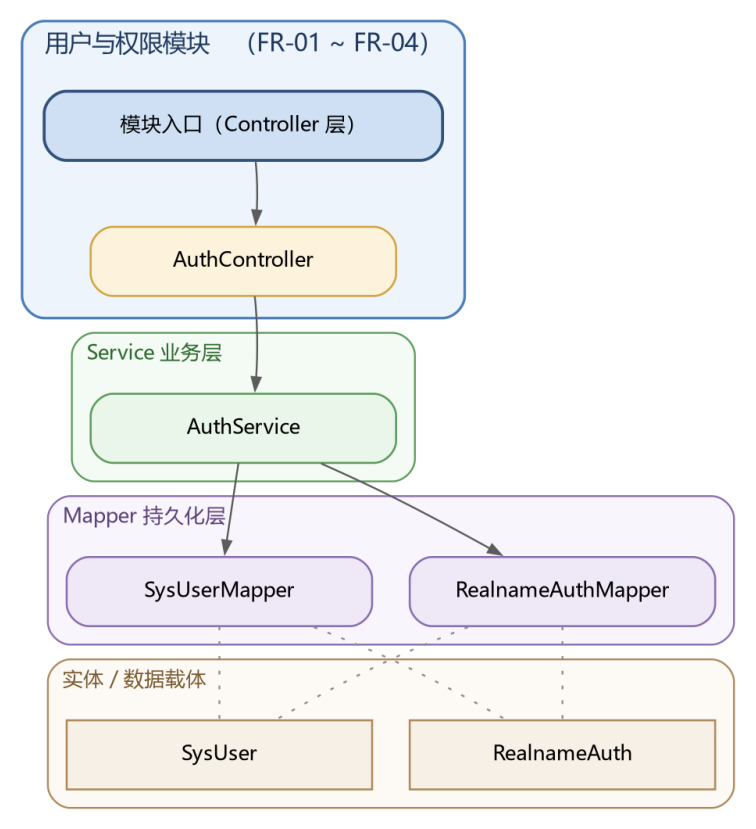


图 3-1 用户与权限模块结构图

模块对外接口函数清单如下表所示，函数级规约见第 5.2 节。

表 3-3 用户与权限模块接口说明

序号	模块名称	接口函数	函数说明
1	用户与权限	sendCaptcha(CaptchaReq)	下发注册 / 登录验证码
2	用户与权限	register(RegisterReq)	注册并自动登录，返回 JWT
3	用户与权限	login(LoginReq)	账号密码登录，含失败计数与锁定
4	用户与权限	currentUser()	查询当前登录用户资料
5	用户与权限	updateProfile(UpdateProfileReq)	修改昵称 / 邮箱 / 头像
6	用户与权限	changePassword(ChangePasswordReq)	修改密码（校验原密码）
7	用户与权限	submitRealname(RealnameReq)	提交实名认证（加密存储）
8	用户与权限	realnameOf(long uid)	查询实名认证状态（发布房源前置校验依据）

3.1.3 用户与权限模块类图

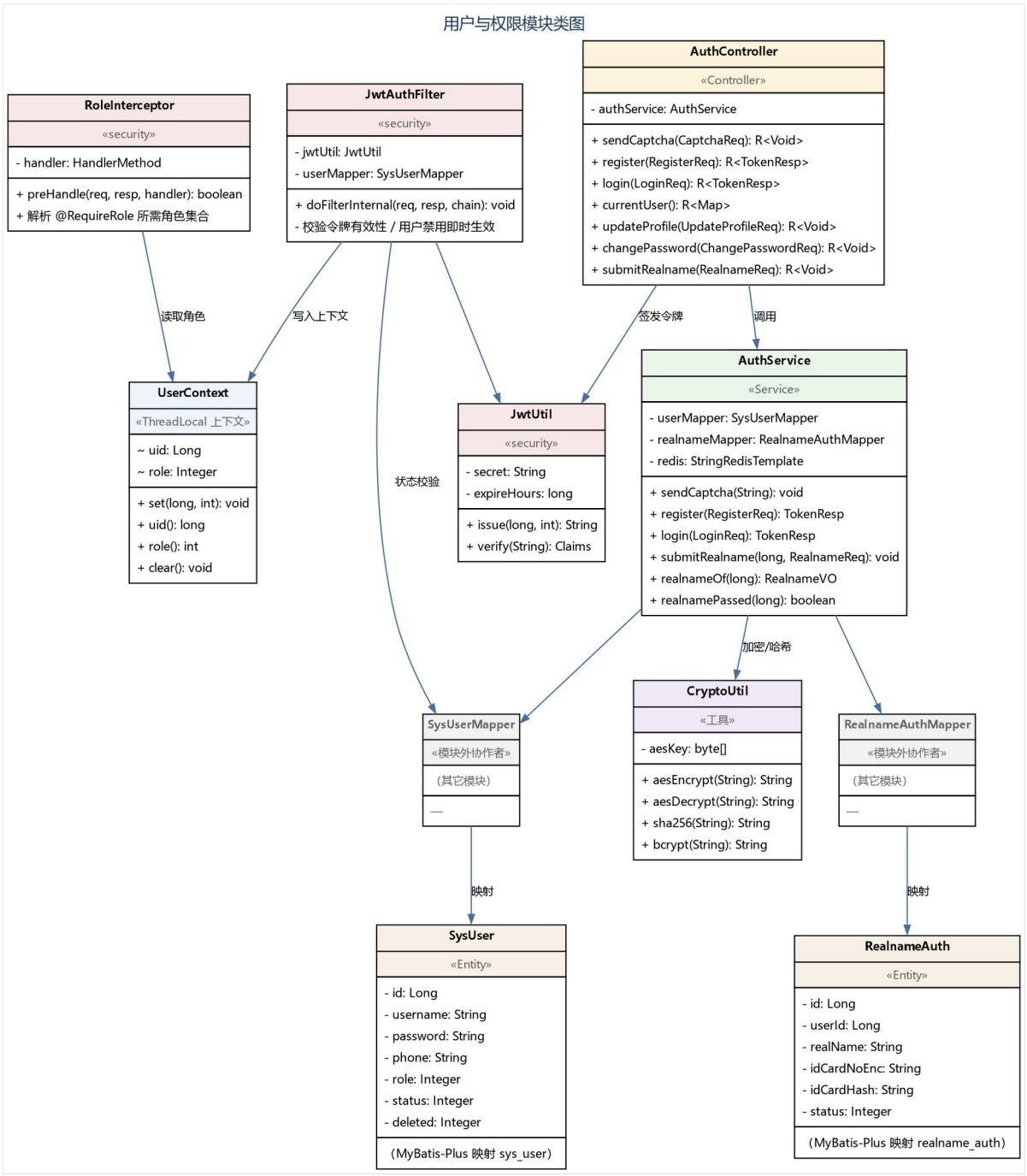


图 3-2 用户与权限模块类图

本模块类图如图 3-2 所示。AuthController 只负责参数校验与响应封装，业务规则集中在 AuthService；JwtUtil、JwtAuthFilter、RoleInterceptor 与 UserContext 构成安全横切组件；CryptoUtil 提供 AES / SHA-256 / BCrypt 三类密码学能力，供实名认证与密码存储复用。

3.2 房源管理模块

房源管理模块负责房源这一核心实体的全生命周期：发布、编辑、上下架、审核，以及配套的图片管理与 AI 信息识别。房源状态按受限状态机流转，任何状态变更都必须携带期望前态，杜绝非法跳转。

审核是平台质量的关键环节：房东发布与编辑后的房源均进入待审核状态，只有审核通过且已上架的房源才进入检索域，供租客检索与推荐使用。

3.2.1 房源管理模块功能定义

表 3-4 房源管理模块功能定义

序号	功能点	功能点详细说明
1	房源发布	房东填写标题、小区、城市、行政区、地址、户型、面积、朝向、楼层、租金、押金方式、设施、描述、经纬度与图片；发布前校验实名认证已通过；初始状态为待审核（FR-05）
2	房源编辑	房东编辑自有房源，编辑后状态回退为待审核，须重新审核方可上架（FR-06）
3	房源上下架	状态机：待审核 → 已通过 / 已驳回 → 已上架 → 已下架 / 已出租；仅“已通过”状态允许上架（FR-06）
4	房源审核	管理员查看待审核列表，执行通过或驳回并填写驳回理由；审核动作全量写入审计日志，结果经站内信通知房东（FR-07）
5	房源信息智能识别	依据标题与图片名关键词推断描述、朝向、楼层与设施，向房东提供一键填充（结果可修改），分析结果异步落 ai_analysis（FR-08）
6	图片管理	图片上传校验文件类型与大小（单文件 ≤ 10MB、单请求 ≤ 30MB），存储于本地图片卷并由 Nginx 托管；支持排序与封面标记（FR-05）
7	房源详情	房东可查看自有全部状态房源；租客侧仅可见已上架房源；详情访问计为热度，作为推荐热度因子（FR-11）——仅非房主、非管理员浏览时累加，避免房东自查与管理端翻台账把房源热度刷高
8	房源状态联动	合同生效后房源自动置为“已出租”并退出检索域（FR-18）

3.2.2 房源管理模块结构

房源管理模块的模块结构如下图所示。

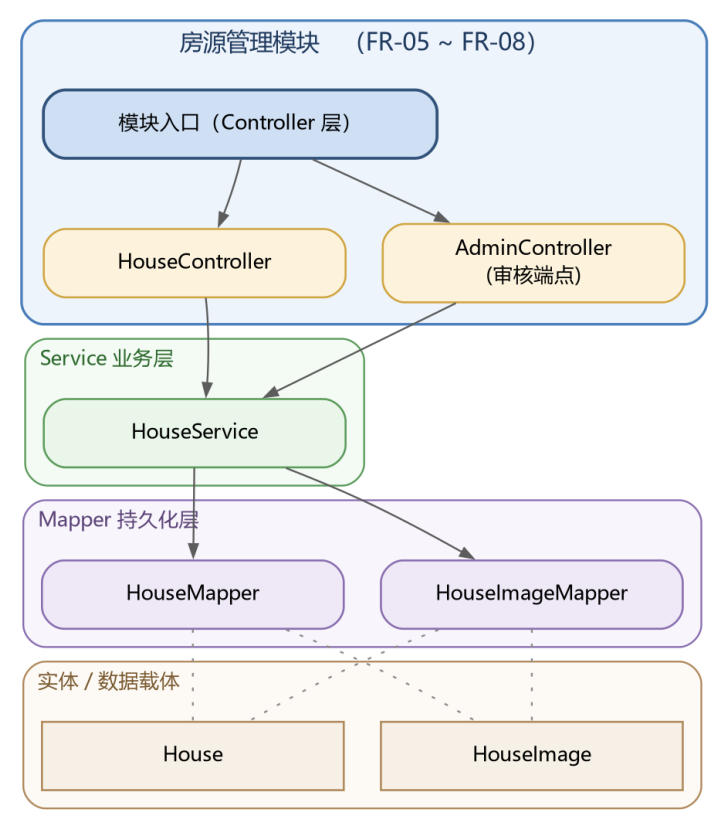


图 3-3 房源管理模块结构图

模块对外接口函数清单如下表所示，函数级规约见第 5.2 节。

表 3-5 房源管理模块接口说明

序号	模块名称	接口函数	函数说明
1	房源管理	create(SaveReq)	发布房源（需实名通过，初始为待审核）
2	房源管理	update(long id, SaveReq)	编辑房源并回退为待审核
3	房源管理	changeStatus(long id, StatusReq)	房源上架 / 下架
4	房源管理	myHouses(page, size)	房东自有房源分页列表
5	房源管理	detail(long id)	房源详情（租客侧过滤非上架房源）
6	房源管理	audit(long id, Map body)	管理员审核：通过 / 驳回（含驳回理由）
7	房源管理	pendingHouses(page, size)	待审核房源分页列表
8	房源管理	assistFill(AiFillReq)	AI 信息识别与一键填充（FR-08）
9	房源管理	upload(MultipartFile)	房源图片上传（限房东 / 管理员）

3.2.3 房源管理模块类图

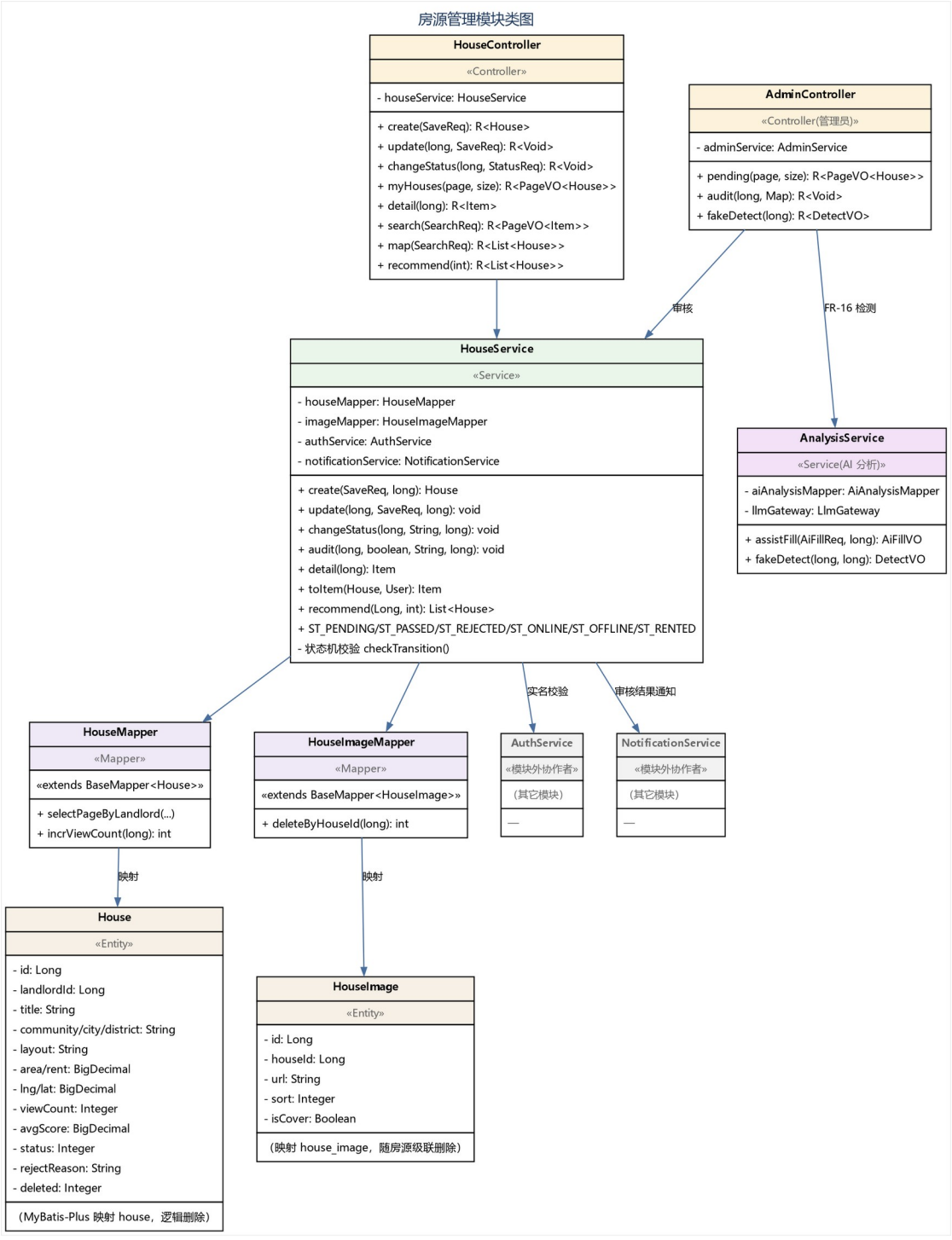


图 3-4 房源管理模块类图

本模块类图如图 3-4 所示。HouseService 集中承载状态机校验与属主校验，并依赖 AuthService 完成实名校验前置、依赖 NotificationService 推送审核结果；AdminController 复用 HouseService 的审核能力，并在审核环节可选调用 AnalysisService 触发虚假房源检测（FR-16）。

3.3 检索与推荐模块

检索与推荐模块面向租客侧的信息获取：关键词与多条件组合筛选、地图找房、个性化推荐，以及收藏管理。本模块的所有查询入口共享同一条硬约束——只返回状态为“已上架”的房源，未审核、已下架与已出租房源一律不可检索。

检索性能依赖组合索引与强制分页（NFR-01）；推荐以收藏与浏览行为作偏好加权，并以浏览热度兜底，在数据稀疏的演示规模下仍可给出稳定结果。

3.3.1 检索与推荐模块功能定义

表 3-6 检索与推荐模块功能定义

序号	功能点	功能点详细说明
1	关键词搜索	对标题、小区、地址做关键词匹配（初期采用 title 索引 + LIKE 前缀匹配，数据量增长后启用全文索引）；仅返回已上架房源（FR-09）
2	多条件组合筛选	支持行政区、户型、朝向、租金区间、设施多选等条件的动态组合，并提供租金升序 / 租金降序 / 热度 / 最新四种排序（FR-09）
3	地图找房	按前端地图视窗的经纬度范围查询房源，不分页返回由前端打点渲染（FR-10）
4	个性化推荐	以用户收藏与浏览行为构建区域 / 户型偏好，加权后排序，并以浏览热度兜底；仅返回已上架房源（FR-11）
5	房源收藏管理	收藏与取消收藏即时生效，已收藏态在详情与列表一致展示；唯一约束防止重复收藏；收藏列表分页展示（FR-25）
6	收藏防重	数据库 UNIQUE(user_id, house_id) 约束与业务层判重双重保障，避免并发重复收藏（FR-25）

3.3.2 检索与推荐模块结构

检索与推荐模块的模块结构如下图所示。

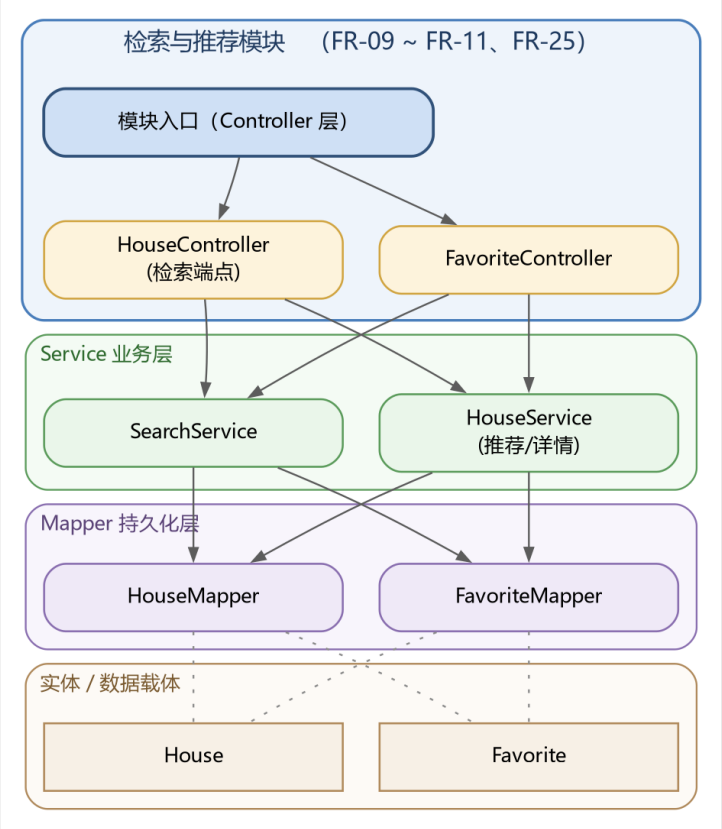


图 3-5 检索与推荐模块结构图

模块对外接口函数清单如下表所示，函数级规约见第 5.2 节。

表 3-7 检索与推荐模块接口说明

序号	模块名称	接口函数	函数说明
1	检索与推荐	search(SearchReq)	关键词与多条件组合筛选分页查询
2	检索与推荐	mapHouses(SearchReq)	地图视窗范围查询（不分页）
3	检索与推荐	recommend(int limit)	个性化推荐（偏好加权 + 热度兜底）
4	检索与推荐	addFavorite(long houseId)	收藏房源
5	检索与推荐	removeFavorite(long houseId)	取消收藏
6	检索与推荐	favorites(page, size)	我的收藏分页列表

3.3.3 检索与推荐模块类图

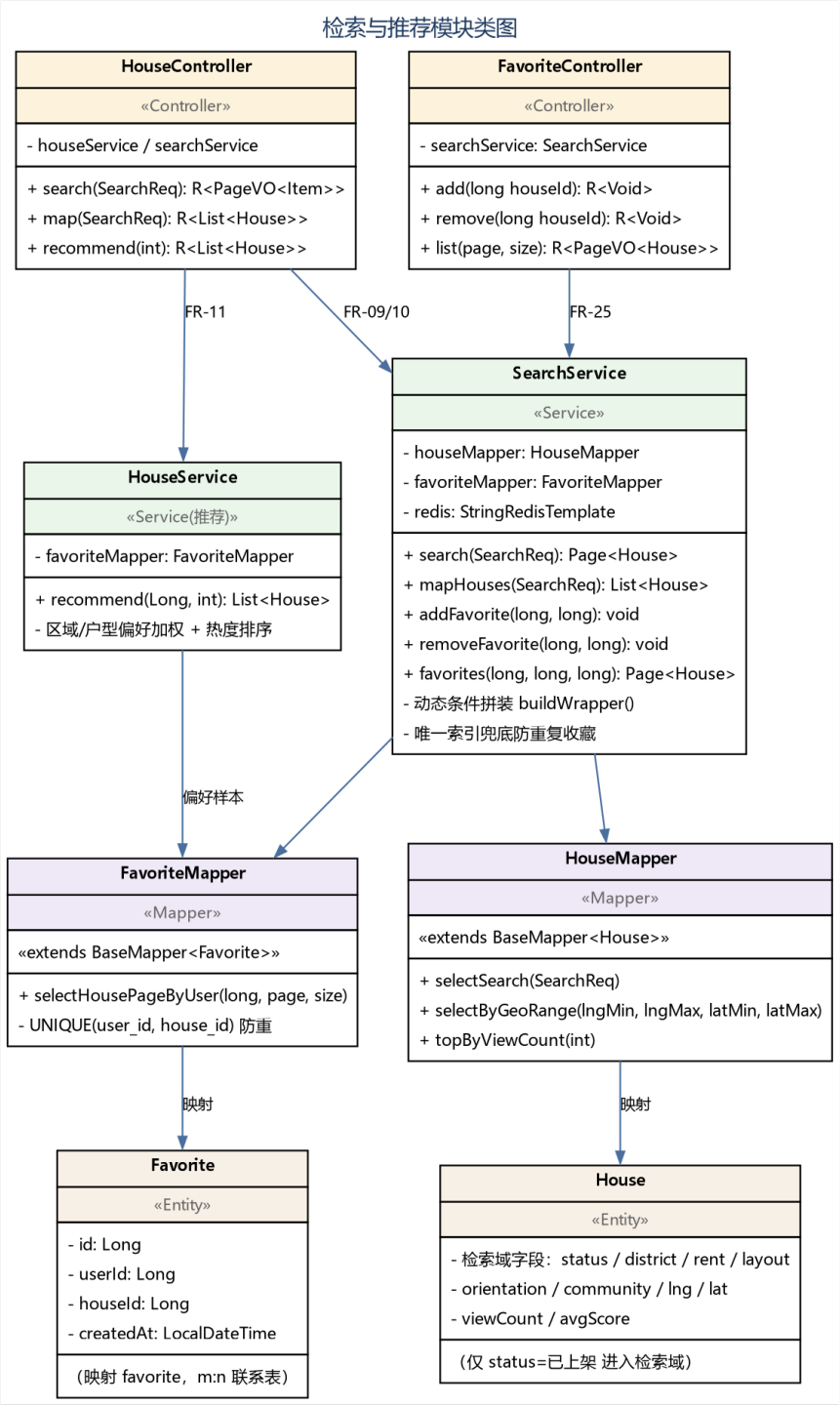


图 3-6 检索与推荐模块类图

本模块类图如图 3-6 所示。检索与收藏分别由 SearchService 统一实现，检索查询收敛在 HouseMapper 的动态条件查询上；推荐能力复用 HouseService，通过 FavoriteMapper 获取偏好样本。

3.4 AI 智能体服务模块

AI 智能体服务模块是本项目的特色模块，覆盖 AI 找房助手、智能客服、合同智能解读、智能定价建议、虚假房源检测与房源信息智能识别六项能力。模块在常规三层架构之下引入智能体层：由 AgentEngine 接口抽象编排能力，LlmAgent 基于 LangChain4j 的 AiServices 实现真实模型编排（流式对话 + Function Calling 工具），四项分析类能力经统一的结构化调用层 AiJsonClient（提示词库 AiPrompts）实现：优先以 response_format=json_schema（strict）由服务端强制约束返回结构，Schema 由目标返回类型经 JsonSchemas 生成，端点不支持该参数时退回“提示词声明字段 + 解析容错”，并对结果做字段与取值域校验。本系统不提供规则引擎或模拟实现：未配置模型、调用失败或返回结构不合规时统一返回 4001 并附可读原因。

模型接入由 LlmGateway 统一收口，支持三种协议后端（openai-chat-completions、anthropic-messages、openai-responses），端点、密钥与模型均可配置，并通过命名为后端的一键切换适配不同供应商。三种协议的适配器均为本项目自研实现：openai-chat-completions 与 anthropic-messages 的适配器对同一轮内的正文与工具调用分别累积后合并（模型普遍“先叙述再调工具”，若按“有正文即丢弃工具调用”处理，工具永远不会被执行），其中 anthropic-messages 适配器还兼容端点的思维链块并主动关闭 thinking 以压缩首字延迟。

内容合规通过两条硬约束保障：知识库问答强制附带来源引用，合同与资金类问题只允许按知识库口径作答并附加“AI 生成，仅供参考”声明；全部对话与工具调用记录落库可追溯（NFR-05）。

3.4.1 AI 智能体服务模块功能定义

表 3-8 AI 智能体服务模块功能定义

序号	功能点	功能点详细说明
1	AI 找房助手	对话式自然语言找房：意图理解 → 条件结构化 → 调用房源检索工具 → 生成带推荐理由的回复；支持多轮追问增量修改条件；采用 SSE 流式输出（FR-12）
2	智能客服	面向平台规则、租赁政策与流程 FAQ 的问答；先检索知识库再生成回答并强制附带来源引用；未命中知识库时礼貌提示转人工（FR-13、NFR-05）
3	合同智能解读	逐条通俗化解释合同条款，对风险条款标红提示，输出附“AI 生成，仅供参考”声明；解读结果把风险条款下标回写 contract.risk_flags（只更新该列，避免整行回写覆盖并发发生的签约/终止状态）（FR-14）
4	智能定价建议	按“同小区 → 同区域同户型”两级取样本并计算均价/中位/最值，把房源字段与样本统计交由模型给出租金区间、依据与补充建议（结果校验含 low ≤ high），样本不足时明确提示（FR-15）

序号	功能点	功能点详细说明
5	虚假房源检测	结合房源文本与同区域租金样本由模型评估风险，输出 0~100 风险分与疑点列表（代码侧校验取值域），管理员保留最终裁决权（FR-16）
6	会话与记忆管理	按场景（1 找房 / 2 客服 / 3 合同解读）路由不同的角色设定与工具集；Redis 维护热会话滑动窗口并对超长对话做摘要压缩；会话与消息全量落库支持回溯（FR-12/13/14、NFR-05）
7	工具注册表	业务能力以 @Tool 注解注册，Agent 运行时按需调用；新增工具只需注册定义与提示词片段，不改动对话主流程（NFR-09）
8	多后端 LLM 网关	三种协议后端可配置，均由自研适配器实现（Function Calling、流式输出、json_schema 结构化输出），经 AI_BACKEND 一键切换；超时 60 秒；未配置模型时统一报 4001（风险 R1）
9	流式输出	SSE 逐字推送 delta 与收尾 done 事件；同一轮内“先叙述再调工具”的响应会把正文与工具调用一并还原，工具执行后继续流式出结论（NFR-02）
10	对话审计与工具留痕	用户 / 助手 / 工具三类消息全量落库；每次工具调用记录工具名、入参、返回与耗时，助手回复记录 token 用量；工具留痕由 HousingToolProvider 以工具提供器方式自动完成（新增 @Tool 自动覆盖），客服回答的结构化来源引用取自 searchKnowledge 返回体；管理员可回放全站会话轨迹（NFR-05、FR-24）

3.4.2 AI 智能体服务模块结构

AI 智能体服务模块的模块结构如下图所示。

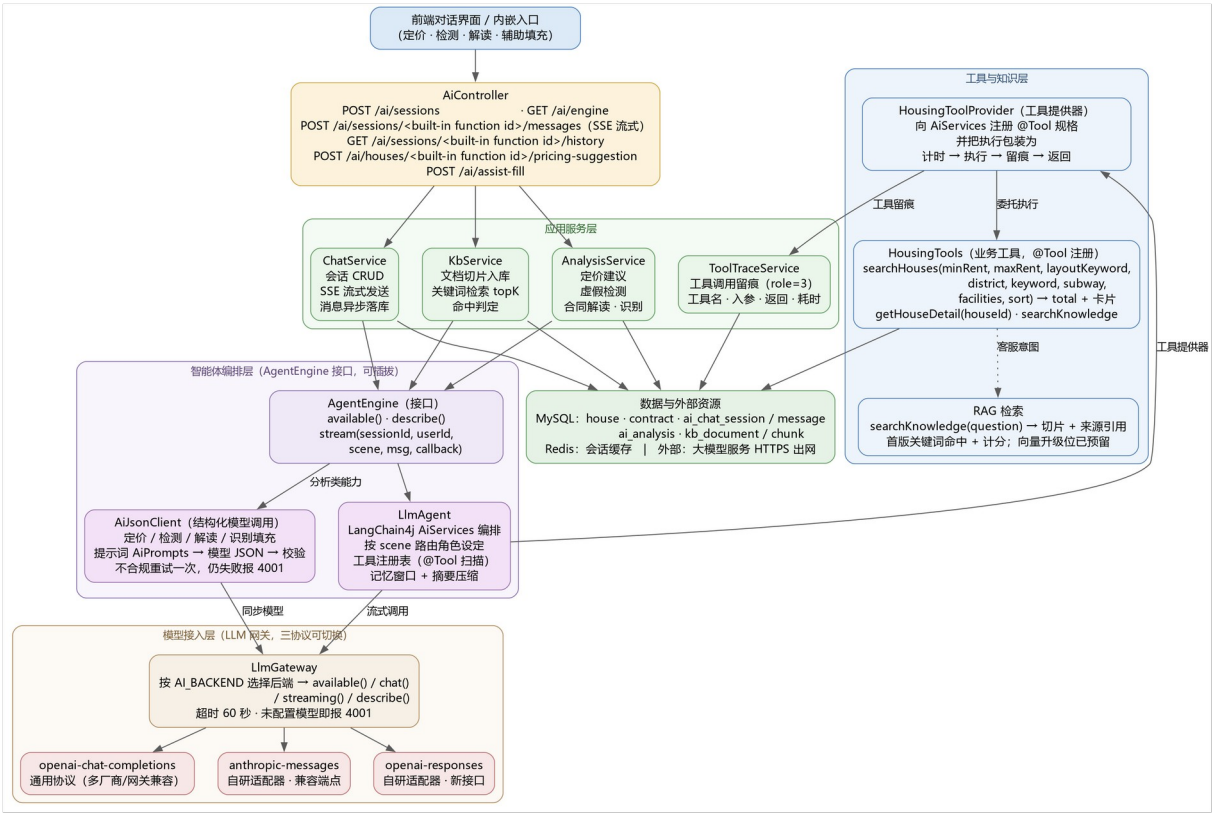


图 3-7 AI 智能体服务模块结构图

模块对外接口函数清单如下表所示，函数级规约见第 5.2 节。

表 3-9 AI 智能体服务模块接口说明

序号	模块名称	接口函数	函数说明
1	AI 智能体服务	createSession(SessionCreateReq)	创建会话（按 scene 路由角色与工具集）
2	AI 智能体服务	send(long id, MessageSendReq)	SSE 流式发送消息（delta* + done）
3	AI 智能体服务	history(long id)	会话历史（含引用与工具调用记录）
4	AI 智能体服务	sessions(page, size)	我的会话分页列表
5	AI 智能体服务	engine()	查询当前生效的模型后端
6	AI 智能体服务	pricing(long houseId)	智能定价建议（FR-15）
7	AI 智能体服务	interpret(long contractId)	合同智能解读（FR-14）
8	AI 智能体服务	fakeDetect(long houseId)	虚假房源检测（FR-16）
9	AI 智能体服务	assistFill(AiFillReq)	房源信息智能识别（FR-08）
10	AI 智能体服务	searchKnowledge(String question)	RAG 知识库检索（工具 / 客服入口）

3.4.3 AI 智能体服务模块类图

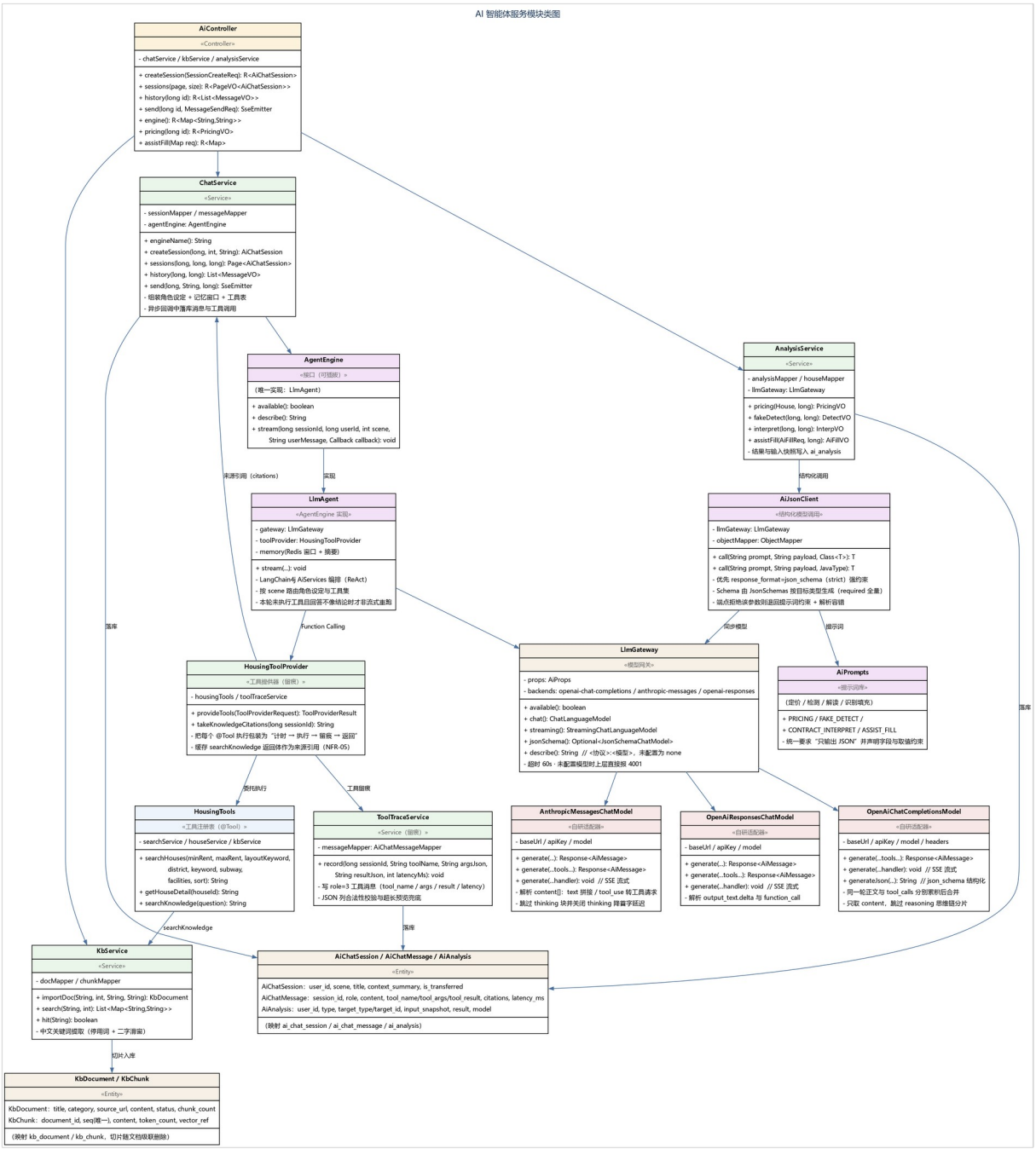


图 3-8 AI 智能体服务模块类图

本模块类图如图 3-8 所示。ChatService 负责会话生命周期与 SSE 发送，KbService 负责知识库切片与检索，AnalysisService 负责四项 AI 分析能力、经 AiJsonClient 完成结构化输出（优先 response_format=json_schema，Schema 由 JsonSchemas 按目标类型生成），提示词集中在 AiPrompts；对话能力经 AgentEngine（唯一实现 LlmAgent）编排，由 LlmGateway 完成协议适配与后端选择（三种协议各有一个自研适配器，均支持正文与工具调用同轮并存）。工具调用留痕由 ToolTraceService 统一落库：

HousingToolProvider 以工具提供器方式包装每个 @Tool 方法的执行（新增工具自动获得留痕，NFR-09），并缓存 searchKnowledge 的返回体作为助手消息的来源引用（NFR-05）。

3.5 交易与合同模块

交易与合同模块覆盖租赁交易的完整链路：预约看房 → 电子合同生成 → 双方签署 → 合同生效 → 派生租赁订单 → 按月生成租金账单。预约与合同均以状态机表达生命周期，并以唯一索引配合事务保证幂等，防止重复预约与重复签约。

按《02-项目最终确认书》的范围约定，本模块不对接真实支付渠道，租金账单仅提供“标记已支付”的记录能力，金额仅作展示与统计用途。

3.5.1 交易与合同模块功能定义

表 3-10 交易与合同模块功能定义

序号	功能点	功能点详细说明
1	预约看房	租客对已上架房源提交预约（预约时段 + 备注，时段须晚于当前时间）；房东可确认或拒绝，租客可取消，房东可标记完成；时段互斥由 UNIQUE(active_slot) 唯一键兜底——占位键仅待确认/已确认持有，终态置 NULL 释放时段，故已取消/已拒绝的时段可被重新预约（FR-17）
2	预约通知	预约创建与每次状态变更均经站内信通知相关方（FR-17、FR-21）
3	在线签约	以合同模板与双方信息自动填充生成电子合同，条款集合以 clauses JSON 存储；租客先签署 → 房东确认签署 → 合同生效；双方均可查看合同副本（FR-18）
4	合同风险标注	生成合同时标注风险条款集合（risk_flags JSON），供 AI 合同解读标红与用户提示（FR-14、FR-18）
5	租金订单	合同生效自动派生租赁订单，contract_id 唯一约束保证一对一；订单承载租期、月租与押金信息（FR-19）
6	租金账单计划	按租期逐月生成账单，UNIQUE(lease_order_id, period_no) 保证重复执行不产生重复账单（幂等）；支持标记已支付并记录支付时间（FR-19）
7	状态机与并发控制	预约与合同的状态更新一律带期望前态条件，配合唯一索引与事务，杜绝非法跳转与并发冲突；时段互斥随状态迁移维护——进入终态的预约释放 active_slot，使同一时段可被重新预约（NFR-08）
8	房源状态联动	合同生效后房源状态置为“已出租”，退出检索域；合同终止后可恢复（FR-18）

3.5.2 交易与合同模块结构

交易与合同模块的模块结构如下图所示。

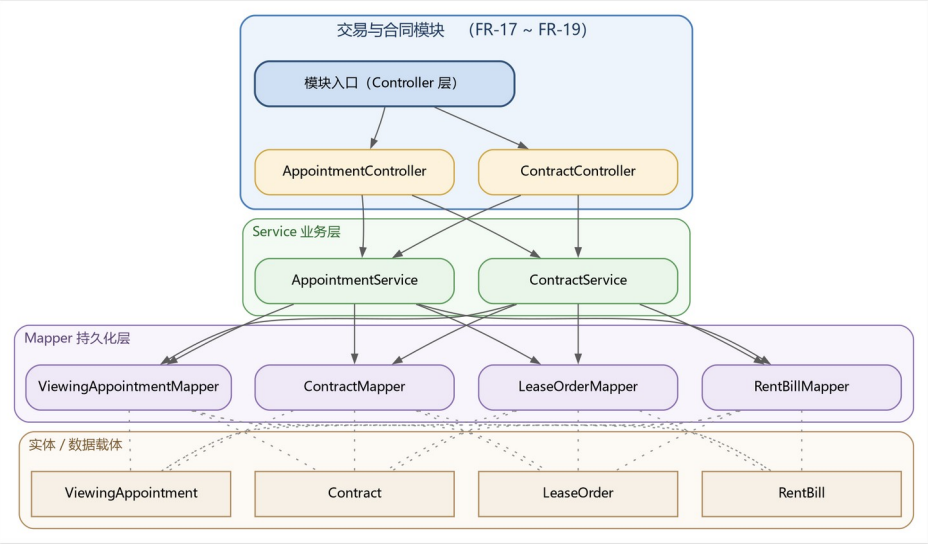


图 3-9 交易与合同模块结构图

模块对外接口函数清单如下表所示，函数级规约见第 5.2 节。

表 3-11 交易与合同模块接口说明

序号	模块名称	接口函数	函数说明
1	交易与合同	createAppointment(AppointmentCreateReq)	创建看房预约
2	交易与合同	actAppointment(long id, AppointmentActionReq)	预约确认 / 拒绝 / 取消 / 完成
3	交易与合同	mine(page, size)	我的预约列表（租客侧）
4	交易与合同	received(page, size)	收到的预约列表（房东侧）
5	交易与合同	createContract(ContractCreateReq)	生成电子合同
6	交易与合同	actContract(long id, ContractActionReq)	合同签署 / 拒签 / 终止
7	交易与合同	detailContract(long id)	合同详情与副本（仅合同双方与管理员可读）
8	交易与合同	orders(page, size)	租赁订单列表
9	交易与合同	bills(long orderId)	租金账单计划
10	交易与合同	payBill(long billId)	标记账单已支付（不对接真实支付）

3.5.3 交易与合同模块类图

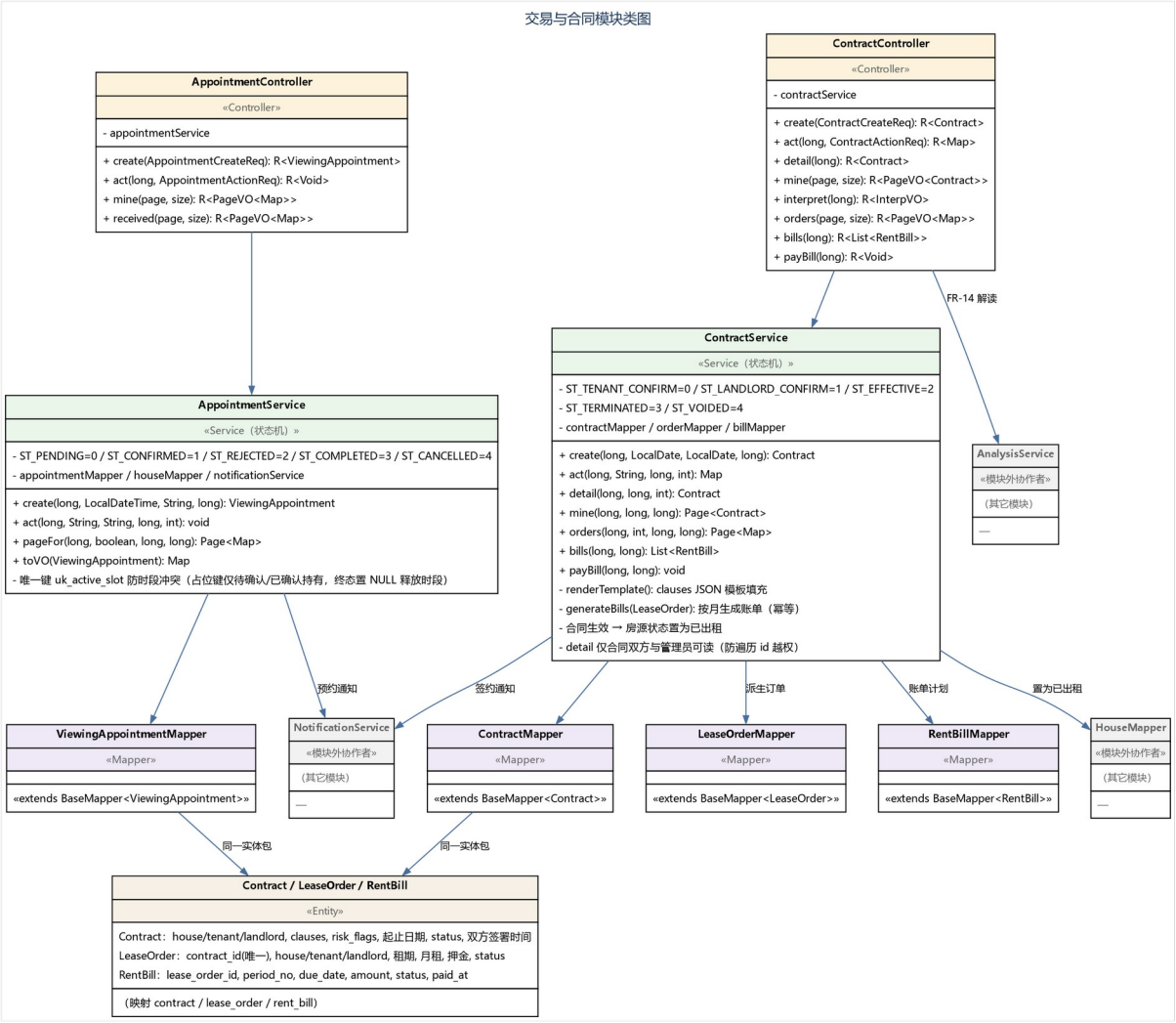


图 3-10 交易与合同模块类图

本模块类图如图 3-10 所示。AppointmentService 与 ContractService 各自维护一条状态机，合同生效的处理链条集中在 ContractService：派生租赁订单 → 生成账单计划 → 置房源为已出租 → 推送双方通知，全部在同一事务内完成。

3.6 评价与消息模块

评价与消息模块承担交易完成后的反馈闭环与全系统的消息触达。评价以租赁订单为唯一锚点，通过唯一约束保证“仅完成合同可评、一单一评”，并在提交后刷新房源的冗余均分。

消息通知采用统一出口：审核结果、预约状态变更、签约、账单生成与举报处理等关键事件均经 NotificationService 写入站内信，前端以轮询方式在 5 秒内可见（FR-21）。

3.6.1 评价与消息模块功能定义

表 3-12 评价与消息模块功能定义

序号	功能点	功能点详细说明
1	评价提交	仅已完成合同的用户可评价，UNIQUE(lease_order_id) 保证一单一评；房源分与房东分取值 1~5，支持文字内容（FR-20）
2	评价展示	按房源分页展示评价列表（含评价人与时间）；被管理员处理为违规的评价不对外展示（FR-20）
3	房源均分刷新	评价提交后刷新 house.avg_score 冗余字段，供列表展示与推荐排序直接使用，避免高频关联聚合（FR-20、NFR-01）
4	站内通知	关键业务事件统一经 NotificationService 触达，通知记录含类型、标题、内容与业务关联（ref_type / ref_id），供前端跳转（FR-21）
5	未读与已读	提供未读数量统计与逐条已读标记，支撑前端消息红点提示（FR-21）
6	举报受理入口	用户对房源或评价提交举报（多态关联 target_type / target_id），进入后台管理的处理队列（FR-23）

3.6.2 评价与消息模块结构

评价与消息模块的模块结构如下图所示。

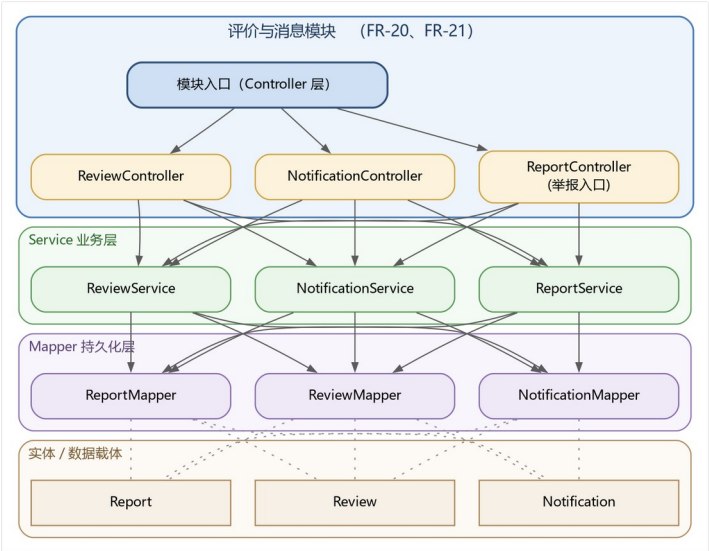


图 3-11 评价与消息模块结构图

模块对外接口函数清单如下表所示，函数级规约见第 5.2 节。

表 3-13 评价与消息模块接口说明

序号	模块名称	接口函数	函数说明
1	评价与消息	create(ReviewReq)	提交评价（校验已完成合同且一单一评）
2	评价与消息	list(long houseId, page, size)	房源评价分页列表
3	评价与消息	send(用户, 类型, 标题, 内容, 关联)	写入站内通知（系统内部接口）
4	评价与消息	list(onlyUnread, page, size)	我的通知分页列表
5	评价与消息	unreadCount()	未读通知数量
6	评价与消息	read(long id)	标记通知已读
7	评价与消息	create(ReportReq)	提交举报

3.6.3 评价与消息模块类图

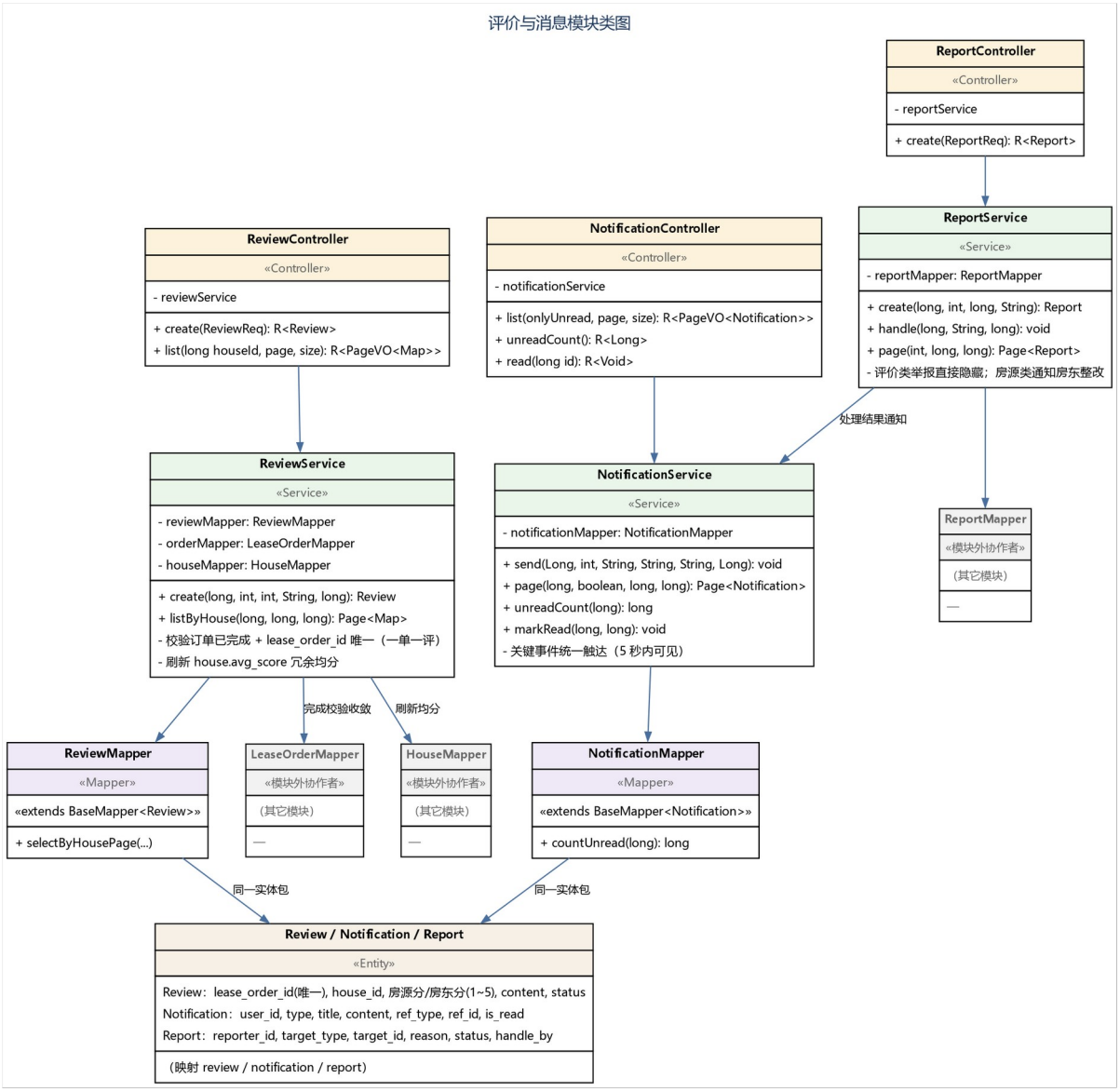


图 3-12 评价与消息模块类图

本模块类图如图 3-12 所示。ReviewService 通过 LeaseOrderMapper 完成“订单已完成”的前置校验，并在同一事务内刷新房源均分；NotificationService 作为全系统消息出口被交易、房源、后台等模块复用，属共享服务而非私有实现。

3.7 后台管理模块

后台管理模块面向管理员角色，集中承载平台治理能力：用户管理、房源审核、举报处理、审计留痕与数据统计看板。整个模块受角色拦截器保护，仅角色 3（管理员）可访问。

可追溯性是本模块的设计重点：审核、禁用、处理举报等管理动作全部写入 audit_log，记录操作人、动作、目标、详情快照与来源 IP，满足 NFR-08 的可维护性要求。

3.7.1 后台管理模块功能定义

表 3-14 后台管理模块功能定义

序号	功能点	功能点详细说明
1	用户管理	用户分页查询（支持关键词）；禁用 / 启用账号，禁用后经每请求状态校验即时生效（FR-22）
2	房源审核管理	待审核房源列表与审核通过 / 驳回（驳回理由必填）；房源与举报的审核结果均通知相关方（FR-07、FR-23）
3	举报处理	按处理状态筛选举报列表；处理意见落库；评价类举报直接隐藏被举报评价，房源类举报通知房东限期整改（FR-23）
4	操作审计留痕	审核、禁用、处理举报等管理动作全量写入审计日志（操作人、动作、目标类型与 ID、详情 JSON、IP）（FR-23、NFR-08）
5	数据统计看板	用户、房源、订单、预约等核心指标的总量与按日 / 周 / 月分组的趋势统计，并含 AI 对话量（会话数、消息数、工具调用数、转人工会话数与会话新增趋势）；全部为只读聚合查询，不写业务表（FR-24）
6	虚假房源检测入口	管理员可在审核环节触发 AI 虚假房源检测，将风险分与疑点作为裁决参考（FR-16）
7	审计日志查询	按时间倒序分页查询审计日志，供问题追溯（FR-23）
8	AI 对话审计	查看全站 AI 会话列表（含归属用户与角色、轮次、工具调用次数、token 与会话标题），并可回放单会话完整轨迹：人物信息、多轮对话、工具调用（工具名 / 入参 / 返回 / 耗时）、RAG 引用来源与角色设定原文；只读，不提供修改与删除入口（NFR-05、FR-24）

3.7.2 后台管理模块结构

后台管理模块的模块结构如下图所示。

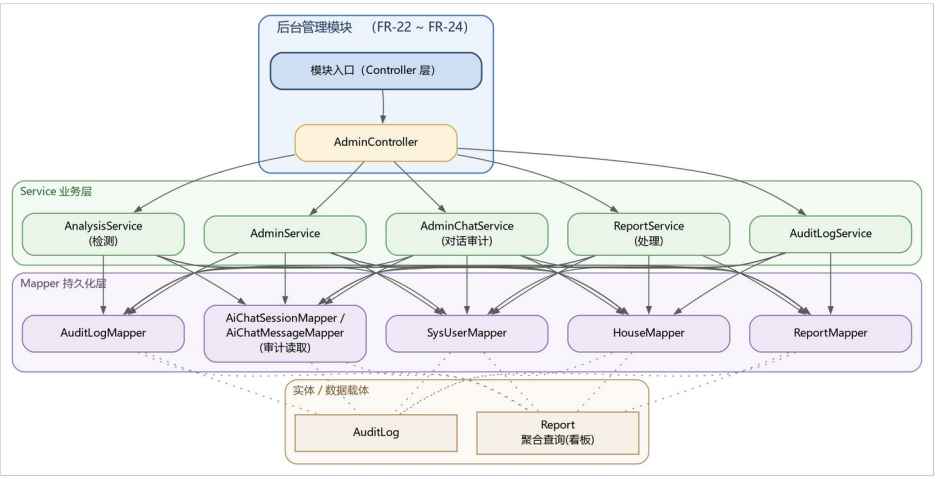


图 3-13 后台管理模块结构图

模块对外接口函数清单如下表所示，函数级规约见第 5.2 节。

表 3-15 后台管理模块接口说明

序号	模块名称	接口函数	函数说明
1	后台管理	users(String keyword, page, size)	用户分页查询
2	后台管理	setUserStatus(long id, boolean enabled)	禁用 / 启用用户（即时生效）
3	后台管理	pendingHouses(page, size)	待审核房源列表
4	后台管理	audit(long id, Map body)	房源审核通过 / 驳回
5	后台管理	fakeDetect(long id)	触发虚假房源检测（FR-16）
6	后台管理	reports(int status, page, size)	举报列表
7	后台管理	handleReport(long id, Map body)	处理举报
8	后台管理	auditLogs(page, size)	审计日志查询
9	后台管理	dashboard(String granularity)	统计看板（day / week / month，含 AI 对话量与会话趋势）
10	后台管理	chats(keyword, scene, transferred, userId, page, size)	AI 会话审计列表（全站，支持关键词 / 场景 / 转人工 / 用户过滤）
11	后台管理	chatDetail(long id)	AI 会话详情（多轮消息 + 工具调用轨迹 + 人物与角色设定）

3.7.3 后台管理模块类图

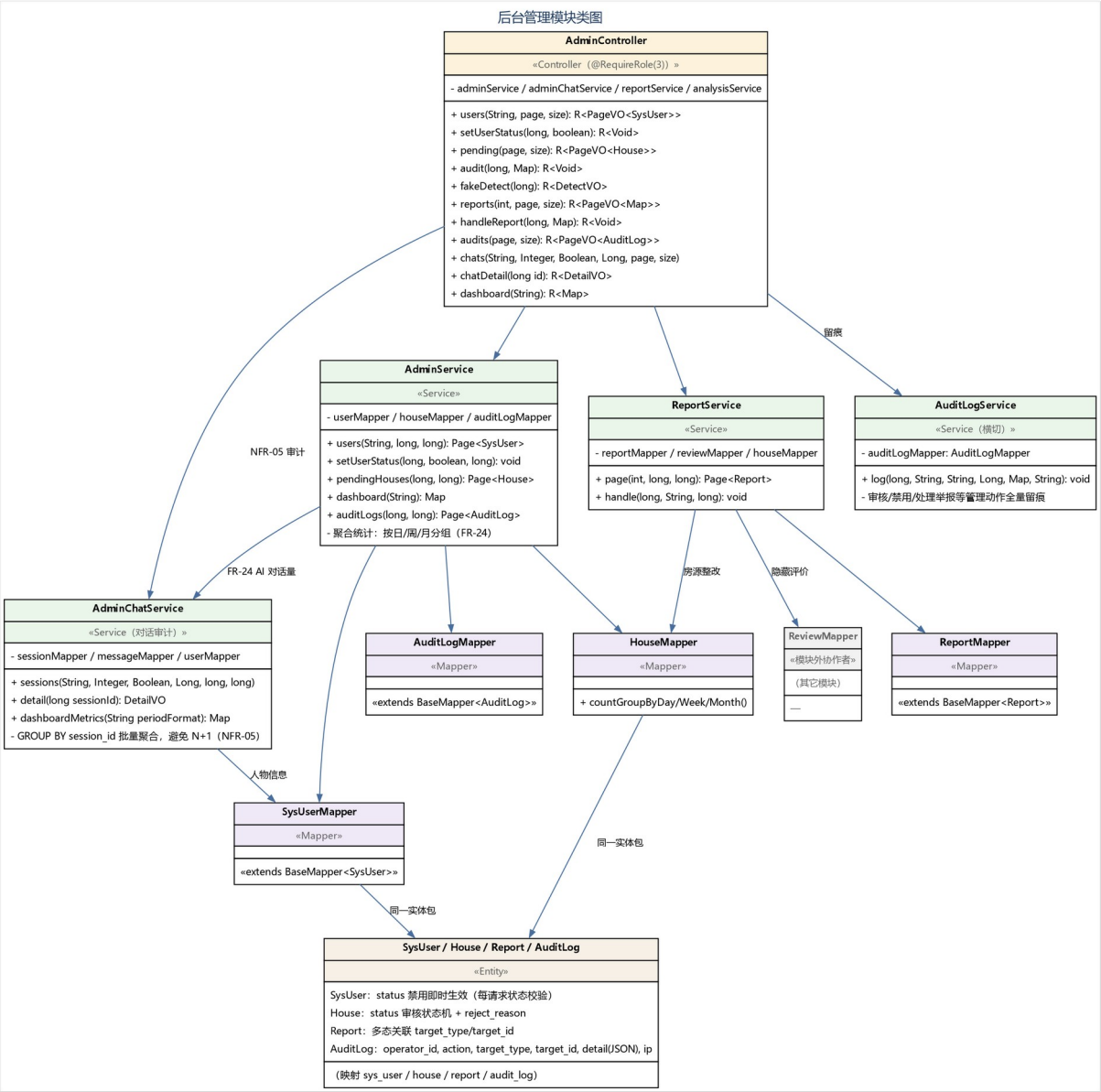


图 3-14 后台管理模块类图

本模块类图如图 3-14 所示。AdminService 承担用户管理、审核与看板聚合，AdminChatService 承担 AI 对话审计的读模型（会话列表与会话轨迹，按会话聚合以避免逐会话查询），ReportService 承担举报流转，AuditLogService 作为横切服务被审核、禁用与举报处理共同调用；看板统计通过聚合查询实现，不引入额外统计表，避免数据冗余与一致性问题。

4 数据库设计

4.1 数据库引擎概述

存储引擎：MySQL 8.0，全部业务表使用 InnoDB，以支持签约、预约等跨表操作的事务一致性；事务隔离级别采用数据库默认的 REPEATABLE READ。

字符集与排序规则：utf8mb4 / utf8mb4_0900_ai_ci，保证中文、生僻字与 emoji 昵称的完整存取。

数据类型约定：主键统一为 BIGINT UNSIGNED 自增；金额一律使用 DECIMAL(10,2)，禁用 FLOAT / DOUBLE；时间使用 DATETIME（东八区）；多值属性与配置快照使用 JSON 类型；长文本使用 MEDIUMTEXT。

外键策略：核心交易链路（realname_auth、notification、house、house_image、favorite、viewing_appointment、contract、lease_order、rent_bill、review、kb_chunk）建立物理外键以保证参照完整性；多态关联与日志类表（audit_log、ai_analysis、report）因无法表达外键关系，采用逻辑外键加业务层校验。

删除策略：业务主数据（sys_user、house）逻辑删除（deleted 字段，MyBatis-Plus 全局逻辑删除）；从属数据（house_image、kb_chunk）随主数据级联物理删除；日志类数据（audit_log、ai_chat_message）只增不改。

并发控制：预约创建与合同签署以“唯一索引 + 事务”兜底幂等；状态更新一律携带期望前态条件（形如 WHERE status = 期望前态），以乐观状态机杜绝非法跳转。

配套存储：Redis 7 承担验证码（5 分钟 TTL）与登录失败计数（10 分钟锁定）两项已实现能力；Redis Stack（RediSearch）的向量索引作为 RAG 语义检索的升级位，kb_chunk.vector_ref 已预留引用字段。

备份与恢复：mysqldump 每日全量备份并同步图片目录、保留 7 份副本；恢复脚本纳入交付物，演练恢复时长不超过 1 小时（NFR-10）。向量数据为派生数据，可由 kb_chunk.content 批量重建，不纳入备份关键路径。

建库方式：数据库结构由 docker/mysql-init/01_schema.sql 一键建库（含全部外键与中文注释），可导入任意 MySQL 8 客户端直接生成数据库；本节的表间关系图即由该脚本的实体与外键关系生成。

4.2 数据库概要设计

本系统共设计 18 张业务表，按业务域组织为四组：用户域（sys_user、realname_auth、notification），房源域（house、house_image、favorite），交易域（viewing_appointment、contract、lease_order、rent_bill、review、report、audit_log），AI 域（ai_chat_session、ai_chat_message、ai_analysis、kb_document、kb_chunk）。

核心实体联系如下：

（1）用户与实名认证、站内通知、房源、预约、合同、举报、审计日志、AI 会话与分析结果均为一对多联系；

（2）用户与房源之间的收藏联系为多对多，通过独立表 favorite 落地，并以 UNIQUE(user_id, house_id) 防重；

（3）房源与房源图片为整体—部分联系，图片随房源级联删除；

（4）合同与租赁订单为一对一联系，以 lease_order.contract_id 唯一约束落地；

（5）租赁订单与租金账单为一对多，以 UNIQUE(lease_order_id, period_no) 保证账单计划幂等；

（6）租赁订单与评价为一对零或一，以 review.lease_order_id 唯一约束实现“一单一评”；

（7）知识库文档与知识库切片为一对多，以 UNIQUE(document_id, seq) 定位切片顺序；

（8）审计日志与 AI 分析结果通过 target_type + target_id 多态关联业务对象，为逻辑外键。

数据库表间关系如图 4-1 所示。图中 PK 表示主键、FK 表示外键，标注为业务表物理外键；标注“逻辑关联”的列不建物理外键，由业务层保证一致性。

18 张业务表的清单与关键字段如表 4-1 所示，字段级完整数据字典见《数据库设计简介》第 4.3 节。

全局物理约定如表 4-2 所示，关键索引设计如表 4-3 所示。索引设计围绕三条主要访问路径：检索域的组合筛选（status + 行政区 + 租金 / 户型）、房东与租客侧的个人列表查询、以及以唯一索引表达的各类业务约束。

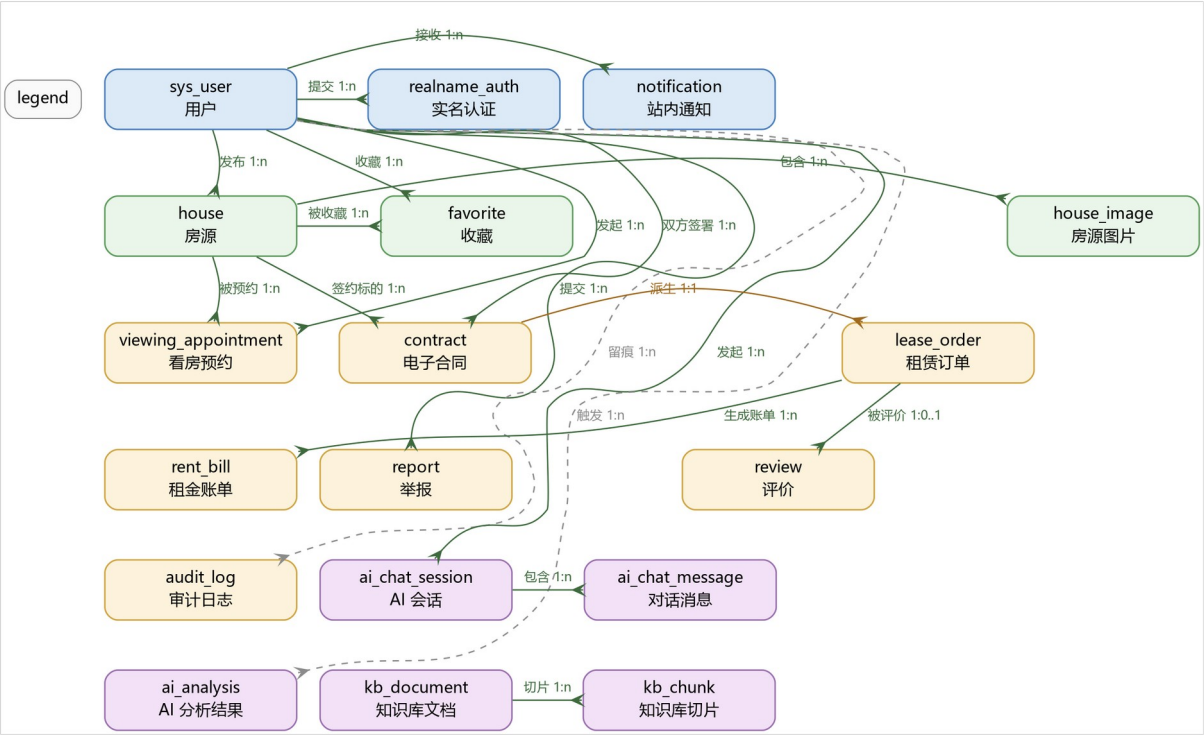


图 4-1 数据库表间关系图

表 4-1 业务表清单与关键字段

序号	表名	中文名	主键	外键	关键字段与约束
1	sys_user	用户	id	—	username、phone 唯一；password(BCrypt)；role 1/2/3；status；deleted
2	realname_auth	实名认证	id	user_id	real_name；id_card_no_enc(AES)；id_card_hash(比对)；status
3	notification	站内通知	id	user_id	type；title；content；ref_type；ref_id；is_read
4	house	房源	id	landlord_id	标题/小区/行政区/户型/面积/租金；facilities(JSON)；lng/lat；status；view_count；avg_score；deleted
5	house_image	房源图片	id	house_id	url；sort；is_cover（随房源级联删除）
6	favorite	收藏	id	user_id、house_id	UNIQUE(user_id, house_id) 防重复收藏
7	viewing_appointment	看房预约	id	house_id、tenant_id、landlord_id	appointment_time；status 状态机；active_slot 时段占位键（唯一键 uk_active_slot 防时段冲突，终态置 NULL 释放）
8	contract	电子合同	id	house_id、tenant_id、landlord_id	clauses(JSON 条款)；risk_flags(JSON)；租期；monthly_rent；deposit；status；双方签署时间
9	lease_order	租赁订单	id	contract_id(唯一)、house_id、tenant_id、landlord_id	start_date/end_date；monthly_rent；deposit；status（1:1 由唯一索引表达）

序号	表名	中文名	主键	外键	关键字段与约束
10	rent_bill	租金账单	id	lease_order_id	period_no；due_date；amount；status；paid_at；UNIQUE(lease_order_id, period_no)
11	review	评价	id	lease_order_id(唯一)、house_id、landlord_id、tenant_id	house_score、landlord_score 取值 1~5；content；status（一单一评）
12	report	举报	id	reporter_id	target_type、target_id（多态逻辑关联）；reason；status；handle_by；handled_at
13	audit_log	审计日志	id	operator_id（逻辑关联）	action；target_type、target_id；detail(JSON)；ip（只增不改）
14	ai_chat_session	AI 会话	id	user_id	scene 1 找房/2 客服/3 合同解读；title；context_summary；is_transferred
15	ai_chat_message	AI 对话消息	id	session_id	role 1 用户/2 助手/3 工具调用；content；tool_name、tool_args、tool_result；citations(JSON)；token_count、latency_ms
16	ai_analysis	AI 分析结果	id	user_id（逻辑关联）	type 定价/检测/解读/识别；target_type、target_id；input_snapshot；result(JSON)；model
17	kb_document	知识库文档	id	—	title；category；source_url；content；status；chunk_count
18	kb_chunk	知识库切片	id	document_id	seq；content；token_count；vector_ref；UNIQUE(document_id, seq)

表 4-2 全局物理约定

项目	约定	依据
存储引擎	InnoDB（全表）	签约、预约等操作需跨表事务一致性
字符集 / 排序	utf8mb4 / utf8mb4_0900_ai_ci	中文、生僻字与 emoji 昵称
主键	BIGINT UNSIGNED 自增	统一主键形态，便于后续分库扩展
金额	DECIMAL(10,2)	禁用浮点类型，保证金额精度
时间	DATETIME（东八区）	统一时区口径，应用层 Jackson 同步配置
外键	核心交易链路建物理外键；多态 / 日志类为逻辑外键	兼顾参照完整性与高频写入性能
删除	主数据逻辑删除；从属数据级联物理删除；日志只增不改	可维护性与审计要求
并发	唯一索引 + 事务；状态更新带期望前态条件	幂等与乐观状态机

表 4-3 关键索引设计

表名	索引	类型	设计依据
sys_user	username、phone	唯一	登录查询与注册查重（FR-01/02）
house	(status, district, rent)、(status, layout)	普通	检索域组合筛选的最左前缀（FR-09）
house	(lng, lat)、title	普通 / 全文	地图找房范围查询（FR-10）、关键词检索（FR-09）

表名	索引	类型	设计依据
favorite	(user_id, house_id)	唯一	防重复收藏（FR-25）
viewing_appointment	active_slot	唯一	预约时段防冲突（FR-17）：占位键 house_id:appointment_time，仅待确认/已确认持有，终态置 NULL 释放时段
lease_order	contract_id	唯一	合同与订单一对一约束（FR-19）
rent_bill	(lease_order_id, period_no)	唯一	账单计划幂等（FR-19）
review	lease_order_id	唯一	一单一评（FR-20）
notification	(user_id, is_read, created_at)	普通	未读通知列表（FR-21）
ai_chat_message	(session_id, id)	普通	会话历史按序读取（FR-12 回溯）
kb_chunk	(document_id, seq)	唯一	切片顺序定位（FR-13）

5 接口设计

5.1 全局变量

本项目为 Java 应用，不存在 C / C++ 意义上的全局变量。原模板中“全局变量”所承载的全局约定，在本项目中由三类元素承担：以常量类集中维护的业务状态字典、以配置文件与环境变量注入的运行期配置项、以及以 ThreadLocal 维护的请求级用户上下文。本节分别说明。

其一，业务状态字典。房源状态、预约状态、合同状态等以静态常量集中定义，与数据库 TINYINT 字段一一对应，避免魔法数字散落；说明见下表。

表 5-1 业务状态字典

类别	标识	取值与含义	用途
用户角色	role	1 = 租客，2 = 房东，3 = 管理员	sys_user.role
用户状态	status	0 = 禁用，1 = 正常	sys_user.status
实名认证状态	realname_auth.status	0 = 待审核，1 = 已通过，2 = 已驳回	FR-03 前置条件
房源状态	HouseService.ST_*	0 待审核 / 1 已通过 / 2 已驳回 / 3 已上架 / 4 已下架 / 5 已出租	FR-06/07 状态机
预约状态	AppointmentService.ST_*	0 待确认 / 1 已确认 / 2 已拒绝 / 3 已完成 / 4 已取消	FR-17 状态机
合同状态	ContractService.ST_*	0 待租客确认 / 1 待房东确认 / 2 已生效 / 3 已终止 / 4 已作废	FR-18 状态机

类别	标识	取值与含义	用途
账单状态	rent_bill.status	0 = 未支付，1 = 已支付（仅记录，不对接支付渠道）	FR-19
AI 会话场景	ai_chat_session.scene	1 = 找房助手，2 = 智能客服，3 = 合同解读	FR-12/13/14 路由
AI 消息角色	ai_chat_message.role	1 = 用户，2 = 助手，3 = 工具调用（role=3 由工具留痕服务写入）	NFR-05 留痕
通知类型	notification.type	1 审核 / 2 预约 / 3 签约 / 4 账单 / 5 举报处理	FR-21

其二，错误码字典。统一响应体 R<T> 的 code 字段以分段方式定义，0 表示成功，非 0 按业务域分段：1xxx 用户与权限、2xxx 房源、3xxx 交易、4xxx AI、5xxx 系统。错误码集中定义于 ErrorCode 枚举，由全局异常处理器统一转换为响应体，前端按段位做差异化处理。

表 5-2 业务错误码

错误码	枚举名	提示信息
1000	PARAM_INVALID	参数校验失败
1001	ACCOUNT_EXISTS	该手机号已注册，请直接登录
1002	LOGIN_FAILED	账号或密码错误
1003	ACCOUNT_LOCKED	密码错误次数过多，账号已锁定 10 分钟
1004	ACCOUNT_DISABLED	该账号已被禁用
1005	CAPTCHA_INVALID	验证码错误或已过期
1006	USER_NOT_FOUND	用户不存在
1007	FORBIDDEN	无权限执行该操作
1008	REALNAME_REQUIRED	请先完成实名认证
2001	HOUSE_NOT_FOUND	房源不存在或已删除
2002	HOUSE_STATUS_INVALID	房源当前状态不允许该操作
2003	FILE_UPLOAD_FAILED	图片上传失败
3001	APPOINTMENT_CONFLICT	该时段已被预约，请换个时间
3002	APPOINTMENT_STATUS_INVALID	预约当前状态不允许该操作
3003	CONTRACT_STATUS_INVALID	合同当前状态不允许该操作
3005	REVIEW_NOT_ALLOWED	仅完成合同后可评价，且一单一评
4001	AI_UNAVAILABLE	智能助手繁忙，请稍后再试
4002	AI_SESSION_NOT_FOUND	会话不存在
5000	SYSTEM_ERROR	系统繁忙，请稍后再试

其三，运行期配置项。关键运行参数通过 application.yml 定义并支持环境变量覆盖，生产部署时无需改动代码：JWT 密钥与有效期（rentagent.jwt-secret / jwt-expire-hours）、上传目录（rentagent.upload-dir）、验证码演示值（rentagent.captcha-dev-code）、身份证明文加密密钥（rentagent.aes-key）、跨域白名单（rentagent.cors-origins），以及 AI 后端选择与超时（ai.active / ai.timeout-seconds）。

其四，请求级用户上下文。登录用户身份由 JWT 解析后写入 ThreadLocal，全过程显式传递，请求结束后立即清理，避免线程复用导致的身份串号：

```
public final class UserContext {
    private static final ThreadLocal<User> HOLDER = new ThreadLocal<>();

    public static void set(long uid, int role, String nickname) { ... }
    public static long uid()  { ... }   // 当前登录用户 ID
    public static int  role() { ... }   // 1 租客 / 2 房东 / 3 管理员
    public static void clear() { HOLDER.remove(); }
}
```

5.2 模块间接口函数设计

本节按模块列出接口函数的详细规约。函数名统一采用“动词 + 业务对象”命名，同名的重载以参数类型区分；所有函数均以统一响应体 R<T> 返回（SSE 流式接口除外）。

5.2.1 用户与权限模块

用户与权限模块共包含 5 个对外接口函数，规约如下。

函数名	sendCaptcha		
文件名	AuthController.java / AuthService.java		
功能概要	下发注册或登录用的短信验证码，并写入 Redis 设置有效期		
参数			
类型	**变量名**	**I/O**	**说明**
CaptchaReq	req	IN	手机号（11 位数字，正则校验）
返回值	**类型**	OUT	**说明**
	类型		R<Void>
	值		成功：code = 0，message = "ok"，data = null
	值		失败：code = 1000，参数校验失败并返回中文提示
详细说明			
校验手机号格式后生成验证码并写入 Redis 键 captcha:{phone}，设置 5 分钟过期时间；演示期为便于评审使用固定验证码 246810，生产环境接入短信服务商后清除该配置项。			
使用注意事项			
验证码接口为匿名可访问白名单接口；同一手机号重复下发会覆盖旧验证码，避免验证码堆积。			

函数名	register
文件名	AuthController.java / AuthService.java
功能概要	手机号 + 验证码 + 密码注册，注册即选定角色并自动登录

函数名	register		
参数			
类型	**变量名**	**I/O**	**说明**
RegisterReq	req	IN	手机号、验证码、密码（6~32 位）、角色 role（1 租客 / 2 房东）、昵称（可选）
返回值	**类型**	OUT	**说明**
	类型		R<TokenResp>
	值		成功：code = 0，data = { token, userId, role, nickname }
	值		失败：code = 1001 手机号已注册；code = 1005 验证码错误或已过期；code = 1000 参数校验失败
详细说明			
依次执行参数校验、验证码比对（读取 Redis）、手机号唯一性校验；通过后以 BCrypt 哈希密码并写入 sys_user，随后直接签发 JWT 返回，前端据 role 跳转对应角色首页。			
使用注意事项			
注册与登录共用同一账号体系，注册完成后无需再次登录；角色仅允许租客或房东，管理员账号由初始化脚本预置。			

函数名	login		
文件名	AuthController.java / AuthService.java		
功能概要	账号密码登录，含失败计数与锁定策略		
参数			
类型	**变量名**	**I/O**	**说明**
LoginReq	req	IN	username（用户名或手机号）、password
返回值	**类型**	OUT	**说明**
	类型		R<TokenResp>
	值		成功：code = 0，data = { token, userId, role, nickname }
	值		失败：code = 1002 账号或密码错误；code = 1003 错误 5 次锁定 10 分钟；code = 1004 账号已被禁用
详细说明			
按用户名或手机号查询用户，校验账号状态后用 BCrypt 比对密码；比对失败则对 Redis 计数器 login:fail:{username} 自增并设置 10 分钟过期，达到 5 次返回锁定错误；比对成功则清零计数并签发 JWT。			
使用注意事项			
登录失败提示统一为“账号或密码错误”，不区分账号是否存在，避免账号枚举；锁定以账号为维度而非 IP。			

函数名	submitRealname		
文件名	AuthController.java / AuthService.java		
功能概要	提交实名认证，身份证号加密存储并另存哈希用于比对		
参数			
类型	**变量名**	**I/O**	**说明**
long	uid	IN	当前登录用户 ID（取自 UserContext）
RealnameReq	req	IN	realName 真实姓名、idCardNo 身份证号（17 位数字 + 校验位）

函数名	submitRealname		
返回值	类型	OUT	说明
	类型		R<Void>
	值		成功：code = 0，认证记录状态为待审核
	值		失败：code = 1000 身份证号格式不正确
详细说明			
身份证号以 AES 加密后写入 realname_auth.id_card_no_enc，同时计算 SHA-256 写入 id_card_hash 用于同人比对；姓名以明文存储供审核核对。房东发布房源前由 HouseService 调用 realnamePassed 校验。			
使用注意事项			
同一用户可多次提交（驳回后重新提交），认证状态以最新一条记录为准；AI 对话上下文不携带证件号与手机号，满足 NFR-04 的隐私合规要求。			

函数名	currentUser		
文件名	AuthController.java / AuthService.java		
功能概要	查询当前登录用户的资料与实名认证状态		
参数			
类型	变量名	I/O	说明
long	uid	IN	当前登录用户 ID（取自 UserContext）
返回值	类型	OUT	说明
	类型		R<Map>
	值		成功：code = 0，data = { id, username, phone, nickname, email, avatarUrl, role, status, realnameStatus }
	值		失败：code = 1006 用户不存在
详细说明			
按主键查询用户并组装资料视图对象，敏感字段不返回明文（手机号按需掩码）；同时返回实名认证状态，供前端在房东侧给出发布入口的可用性提示。			
使用注意事项			
本接口为登录后各页面的公共依赖，前端在应用初始化时调用一次并缓存于 Context。			

5.2.2 房源管理模块

房源管理模块共包含 5 个对外接口函数，规约如下。

函数名	createHouse		
文件名	HouseController.java / HouseService.java		
功能概要	房东发布房源，初始状态为待审核		
参数			
类型	变量名	I/O	说明
SaveReq	req	IN	标题、小区、城市、行政区、地址、户型、面积、朝向、楼层、租金、押金方式、设施列表、描述、经度、纬度、图片列表

函数名	createHouse		
long	landlordId	IN	当前登录房东 ID（取自 UserContext）
返回值	类型	OUT	说明
	类型		R<House>
	值		成功：code = 0，data 为新建房源（status = 0 待审核）
	值		失败：code = 1008 请先完成实名认证；code = 1000 参数校验失败（面积 / 租金 / 经纬度越界）
详细说明			
先调用 AuthService.realnamePassed 校验房东实名状态；通过后写入 house 主表并将图片列表逐条写入 house_image（首图标记为封面），同时把首图同步写入 house.cover_url 冗余列，全部操作在同一事务内完成。			
使用注意事项			
发布后不可直接被租客检索到，必须先经管理员审核通过并上架；封面冗余列供列表卡片与预约单直接读取，避免逐条回表取图；面积、租金、经纬度在 DTO 层以注解约束取值范围，避免异常数据入库。			

函数名	updateHouse		
文件名	HouseController.java / HouseService.java		
功能概要	房东编辑自有房源，编辑后状态回退为待审核		
参数			
类型	变量名	I/O	说明
long	id	IN	房源 ID
SaveReq	req	IN	同发布接口的字段集合
long	uid	IN	当前登录用户 ID
返回值	类型	OUT	说明
	类型		R<Void>
	值		成功：code = 0，房源状态回退为 0 待审核
	值		失败：code = 2001 房源不存在；code = 2002 房源当前状态不允许该操作（非本人房源 / 已出租）
详细说明			
校验房源存在且 landlord_id 等于当前用户（属主校验防水平越权），并校验当前状态允许编辑；随后更新主表字段，并将状态回退为待审核。图片按“先删后插”方式重建并同步刷新 house.cover_url；请求未携带 images 字段时视为“本次不改动图片”（保留既有图片与封面），传空数组才是显式清空。			
使用注意事项			
编辑即回退审核是保证信息可信度的关键约束：任何对外可见的房源内容都必须经过审核；已出租状态的房源不允许编辑；“不传即不改动”用于列表页等只改文本字段的场景，避免这些请求把已有照片整体删空。			

函数名	changeStatus		
文件名	HouseController.java / HouseService.java		
功能概要	房东对已审核通过的房源执行上架或下架		
参数			
类型	变量名	I/O	说明
long	id	IN	房源 ID
StatusReq	req	IN	action：上架 / 下架

函数名	changeStatus		
long	uid	IN	当前登录用户 ID
返回值	类型	OUT	说明
	类型		R<Void>
	值		成功：code = 0，房源状态置为 3 已上架 或 4 已下架
	值		失败：code = 2002 房源当前状态不允许该操作
详细说明			
校验属主后按状态机判定迁移合法性：仅“已通过”（1）允许上架为“已上架”（3），“已上架”（3）允许下架为“已下架”（4）；更新语句携带期望前态条件，防止并发下非法跳转。			
使用注意事项			
上架是房源进入检索域的唯一入口；下架后房源立即从检索与推荐结果中消失，但既有预约与合同不受影响。			

函数名	auditHouse		
文件名	AdminController.java / HouseService.java		
功能概要	管理员审核房源：通过或驳回并记录驳回理由		
参数			
类型	变量名	I/O	说明
long	id	IN	房源 ID
boolean	pass	IN	true 通过，false 驳回
String	reason	IN	驳回理由（驳回时必填）
long	adminId	IN	当前管理员 ID
返回值	类型	OUT	说明
	类型		R<Void>
	值		成功：code = 0，房源状态置为 1 已通过 或 2 已驳回
	值		失败：code = 2001 房源不存在；code = 1000 驳回未填写理由
详细说明			
校验房源处于待审核状态后更新状态与驳回理由，随后由控制器层写审计日志（操作人、动作、目标、详情快照、IP）并向房东推送审核结果站内信。			
使用注意事项			
审核动作必须留痕，audit_log 记录不可修改；驳回理由会展示给房东，作为修改房源的依据。			

函数名	assistFill		
文件名	AiController.java / AnalysisService.java		
功能概要	房源信息智能识别：依据标题与图片名推断描述、朝向、楼层与设施		
参数			
类型	变量名	I/O	说明
AiFillReq	req	IN	title 标题、community 小区、layout 户型、imageFileName 图片文件名
返回值	类型	OUT	说明
	类型		R<AiFillVO>

函数名	assistFill		
	值		成功：code = 0，data = { description, orientation, floorDesc, facilities[] }
	值		失败：code = 4001 智能助手繁忙，请稍后再试
详细说明			
以标题、小区、户型与图片名中的关键词为输入，经规则与模型组合推理输出结构化建议字段，分析输入与结果写入 ai_analysis 便于追溯；结果仅作填入草稿，房东可自由修改。			
使用注意事项			
该功能为 P2 增强项，不阻塞发布主流程；未配置模型或模型调用失败时返回 4001 并提示，由房东手工填写，不做本地规则兜底。			

5.2.3 检索与推荐模块

检索与推荐模块共包含 4 个对外接口函数，规约如下。

函数名	searchHouses		
文件名	HouseController.java / SearchService.java		
功能概要	关键词与多条件组合筛选房源，强制分页，仅返回已上架房源		
参数			
类型	变量名	I/O	说明
SearchReq	req	IN	keyword 关键词；district 行政区；layout 户型；orientation 朝向；rentMin / rentMax 租金区间；facilities 设施多选；sort 排序（rent_asc / rent_desc / hot / new）；page / size 分页
返回值	类型	OUT	说明
	类型		R<PageVO<House>>
	值		成功：code = 0，data = { list, total, page, size }，list 为房源实体（facilities 为数组）
	值		失败：code = 1000 参数校验失败（如 size 超过上限 50）
详细说明			
以 MyBatis-Plus 条件构造器动态拼装查询条件，固定附加 status = 已上架 与 deleted = 0 两个条件；关键词对标题 / 小区 / 地址做匹配；结果按 sort 参数排序，默认按最新，并强制分页（默认 10 条、上限 50 条）。返回裸房源实体，与 收藏 / 我的房源 / 推荐位保持同一形状（详情接口才返回带图片与房东名的 Item）。			
使用注意事项			
任何检索入口都不得返回非“已上架”房源，该约束在 Service 层统一施加而非依赖调用方；列表查询依赖 (status, district, rent) 与 (status, layout) 组合索引的最左前缀。			

函数名	mapHouses
文件名	HouseController.java / SearchService.java
功能概要	地图找房：按视窗经纬度范围查询房源，不分页返回

函数名	mapHouses		
参数			
类型	**变量名**	**I/O**	**说明**
SearchReq	req	IN	lngMin / lngMax、latMin / latMax 视窗边界（可为空），其余筛选条件同 searchHouses
返回值	**类型**	OUT	**说明**
	类型		R<List<House>>
	值		成功：code = 0，data 为视窗内已上架房源列表（不分页，上限 300 个点）
	值		失败：code = 1000 经纬度范围参数非法
详细说明			
按 lng / lat 范围条件查询已上架房源并一次性返回，由前端地图组件打点渲染；当视窗过大导致结果过多时按数量上限截断，避免单次响应过大。			
使用注意事项			
地图模式不做分页是出于交互需要（地图缩放需要完整点集）；为避免响应膨胀，接口返回字段精简为打点与卡片展示所需的最小集合。			

函数名	recommend		
文件名	HouseController.java / HouseService.java		
功能概要	个性化推荐房源：收藏与浏览偏好加权并以热度兜底		
参数			
类型	**变量名**	**I/O**	**说明**
Long	uid	IN	当前登录用户 ID（可为空，空则退化为纯热度推荐）
int	limit	IN	返回条数，默认 6
返回值	**类型**	OUT	**说明**
	类型		R<List<House>>
	值		成功：code = 0，data 为已上架房源推荐列表
	值		失败：code = 1000 limit 越界
详细说明			
以用户收藏记录提取区域与户型偏好，按偏好匹配度加权，再以 view_count 浏览热度作为兜底权重排序；固定过滤已上架且非本人的房源，避免推荐到自己的房源。			
使用注意事项			
偏好样本稀疏时（新用户、无收藏）自动退化为热度推荐，保证首页推荐位始终有内容；热度榜的 Redis ZSet 为后续演进预留，首版直接以 view_count 排序实现。			

函数名	addFavorite		
文件名	FavoriteController.java / SearchService.java		
功能概要	收藏房源，唯一约束与业务判重双重防重		
参数			
类型	**变量名**	**I/O**	**说明**
long	uid	IN	当前登录用户 ID
long	houseId	IN	房源 ID

函数名	addFavorite		
返回值	类型	OUT	说明
	类型		R<Void>
	值		成功：code = 0，收藏关系建立（重复收藏为幂等成功）
	值		失败：code = 2001 房源不存在
详细说明			
先校验房源存在，随后尝试写入 favorite；若已存在相同 (user_id, house_id) 记录则直接返回成功，保证接口幂等；并发场景由数据库唯一索引兜底并捕获冲突异常转为成功。			
使用注意事项			
重复收藏按幂等处理而不报错，避免用户重复点击产生无意义错误提示；取消收藏对不存在的记录同样按幂等成功处理。			

5.2.4 AI 智能体服务模块

AI 智能体服务模块共包含 5 个对外接口函数，规约如下。

函数名	createSession		
文件名	AiController.java / ChatService.java		
功能概要	创建 AI 会话，按场景路由角色设定与工具集		
参数			
类型	变量名	I/O	说明
SessionCreateReq	req	IN	scene 场景（1 找房助手 / 2 智能客服 / 3 合同解读）、title 标题（可选）
返回值	类型	OUT	说明
	类型		R<AiChatSession>
	值		成功：code = 0，data 为会话对象（含 id 与 scene）
	值		失败：code = 1000 scene 超出 1~3 范围
详细说明			
在 ai_chat_session 中创建会话记录并绑定当前用户与场景；场景决定后续对话的角色设定、可用工具集合与是否启用知识库检索。			
使用注意事项			
多轮记忆不单独建表：会话记忆 = ai_chat_message 的窗口读取 + context_summary 摘要字段；热会话窗口由 Redis 承载并在超长时压缩为摘要，全量消息以 MySQL 为准。			

函数名	sendMessage		
文件名	AiController.java / ChatService.java		
功能概要	发送对话消息，以 SSE 流式返回模型输出并异步落库		
参数			
类型	变量名	I/O	说明
long	id	IN	会话 ID

函数名	sendMessage		
MessageSendReq	req	IN	content 用户消息文本
返回值	**类型**	OUT	**说明**
	类型		SseEmitter（text/event-stream）
	值		delta 事件：{"delta":"逐字文本片段"}，可多次（按事件名 delta 区分，载荷内不带 type 字段）
	值		done 事件：{"messageId":..,"content":"完整正文","citations":[..],"transferred":true|false,"latencyMs":..}
	值		error 事件：{"error":"可读原因"}，模型调用失败时推送，其后不再有 delta
	值		失败：code = 4002 会话不存在；code = 4001 智能助手繁忙（未配置模型或调用失败，非流式响应）
详细说明			
组装角色设定、记忆窗口与工具表后交由 AgentEngine 编排；模型返回的文本增量以 delta 事件逐字推送，工具调用（如房源检索）在服务端执行后回传模型继续生成；结束时推送 done 事件并携带消息 ID 与引用来源。用户消息与助手回复及工具调用记录在异步回调中写入 ai_chat_message；助手消息正文取本轮 delta 的拼接，并以终止事件携带的正文兜底——部分协议流式 delta 全程为空、只在终止事件给完整正文，此时正文还会补走 delta 通道推给前端（否则界面停在空白气泡），落库与推送口径一致。模型既没返回正文也没返回工具调用时（典型成因：推理内容占满 max_tokens、流被截断）按失败上报，绝不伪造回答；限流 / 5xx / 超时 / 空返回等可自愈故障由编排层以非流式重跑一次，4xx 直接报错。			
使用注意事项			
流式链路需关闭中间缓冲以压缩首字延迟（NFR-02）；单次模型调用超时 60 秒。单轮 token 上限（ai.max-tokens，默认 4096）需给推理内容留足余量：实测一轮对话约 1.6k~2.0k tokens，取值过紧会被 finish_reason=length 截断成“既无正文也无工具调用”。客服场景未命中知识库时在 done 事件中返回转人工提示（transferred = true）。			

函数名	searchKnowledge		
文件名	KbService.java / HousingTools.java		
功能概要	RAG 知识库检索：返回最相关的知识切片与来源标识		
参数			
类型	**变量名**	**I/O**	**说明**
String	question	IN	用户问题文本
int	topK	IN	返回切片数量（默认 3）
返回值	**类型**	OUT	**说明**
	类型		List<Map<String,String>>
	值		命中：返回列表，每项含 docId、docTitle（来源）、content（切片正文）
	值		未命中：返回空列表，调用方据此触发转人工提示
详细说明			
对问题做中文关键词提取（去停用词 + 长片段二字滑窗切词），在 kb_chunk 上按命中数计分并取 topK；返回内容携带文档标题作为来源，供回答强制附带引用（NFR-05）。			

函数名	searchKnowledge
使用注意事项	
首版为关键词检索 + 计分实现，向量语义检索为升级位：kb_chunk.vector_ref 已预留，接入 embedding 后替换检索实现即可，接口契约不变；合同与资金类问题只允许按知识库口径作答。	

函数名	suggestPrice		
文件名	AiController.java / AnalysisService.java		
功能概要	智能定价建议：两级取样给出租金区间、依据与样本量		
参数			
类型	**变量名**	**I/O**	**说明**
long	id	IN	房源 ID（草稿或已发布均可）
long	uid	IN	当前登录用户 ID
返回值	**类型**	OUT	**说明**
	类型		R<PricingVO>
	值		成功：code = 0，data = { low, high, avg, basis, sampleCount, note, model }
	值		样本不足：code = 0，sampleCount 偏小并在 note 中说明依据不足
	值		失败：code = 2001 房源不存在
详细说明			
取样顺序为“同小区同户型 → 同区域同户型”两级回退，样本量不足时逐级放宽并如实上报样本量；输出租金区间、均价、计算依据与样本条数，输入快照与结果写入 ai_analysis。			
使用注意事项			
区间与依据由真实模型给出、样本量由代码统计保证；未配置模型时返回 4001。界面需标注样本量与依据；分析仅使用已上架房源作为样本，避免被下架与未审核数据污染。			

函数名	detectFakeHouse		
文件名	AdminController.java / AnalysisService.java		
功能概要	虚假房源检测：输出风险分与疑点列表，供管理员裁决参考		
参数			
类型	**变量名**	**I/O**	**说明**
long	houseId	IN	房源 ID
long	adminId	IN	当前管理员 ID（用于留痕）
返回值	**类型**	OUT	**说明**
	类型		R<DetectVO>
	值		成功：code = 0，data = { riskScore（0~100）, suspicions[], suggestion, model }
	值		失败：code = 2001 房源不存在
详细说明			
结合房源标题、描述与同区域租金样本统计交由模型评估，输出风险分与具体疑点条目（如价格显著偏离、图片与描述不符、信息缺失等），并给出处置建议；结果写入 ai_analysis 供追溯。			

函数名	detectFakeHouse
使用注意事项	
检测结果为辅助判断，管理员保留最终裁决权，任何处置都必须经人工审核动作并留痕；本功能为 P2 增强项；未配置模型时返回 4001，由其人工审核。	

5.2.5 交易与合同模块

交易与合同模块共包含 5 个对外接口函数，规约如下。

函数名	createAppointment		
文件名	AppointmentController.java / AppointmentService.java		
功能概要	租客对已上架房源创建看房预约，唯一索引防时段冲突		
参数			
类型	**变量名**	**I/O**	**说明**
AppointmentCreateReq	req	IN	houseId 房源 ID、appointmentTime 预约时段（须晚于当前时间）、remark 备注
long	tenantId	IN	当前登录租客 ID
返回值	**类型**	OUT	**说明**
	类型		R<ViewingAppointment>
	值		成功：code = 0，data 为新建预约（status = 0 待确认）
	值		失败：code = 3001 该时段已被有效预约占用；code = 2002 房源当前状态不允许预约；code = 1000 预约时间不晚于当前时间
详细说明			
校验房源存在且处于已上架状态、租客非房源本人、预约时间晚于当前时间（秒以下精度先对齐，避免落库值与占位键不一致）；写入预约记录（状态待确认，同时写入时段占位键 active_slot），并通知房东。时段冲突由唯一键 uk_active_slot（UNIQUE(active_slot)）兜底，冲突时转换为 3001 错误码返回。			
使用注意事项			
同一房源同一时段只允许一条仍占用时段的预约（待确认或已确认）：预约进入终态（已拒绝/已完成/已取消）时占位键置 NULL，该时段随之释放、可被重新预约，历史记录仍完整保留；预约本身不锁定房源，仍需房东确认后进入线下看房环节。			

函数名	actAppointment		
文件名	AppointmentController.java / AppointmentService.java		
功能概要	预约状态流转：确认 / 拒绝 / 取消 / 完成		
参数			
类型	**变量名**	**I/O**	**说明**
long	id	IN	预约 ID
AppointmentActionReq	req	IN	action（confirm / reject / cancel / complete）、reason 原因（拒绝时填写）
long	uid	IN	当前登录用户 ID
int	role	IN	当前登录用户角色

函数名	actAppointment		
返回值	类型	OUT	说明
	类型		R<Void>
	值		成功：code = 0，预约状态按状态机迁移
	值		失败：code = 3002 预约当前状态不允许该操作；code = 1007 无权限执行该操作
详细说明			
按角色判定动作权限：房东可 confirm / reject / complete，租客可 cancel；随后按状态机校验前态（如仅“待确认”可被确认或拒绝），更新时携带期望前态条件；状态变更后通知对方相关方。			
使用注意事项			
动作语义按角色区分，服务层需同时校验角色与属主，避免越权操作他人预约；已完成的预约不允许再变更状态。			

函数名	createContract		
文件名	ContractController.java / ContractService.java		
功能概要	以模板与双方信息自动填充生成电子合同		
参数			
类型	变量名	I/O	说明
ContractCreateReq	req	IN	houseId 房源 ID、startDate 租期起、endDate 租期止
long	tenantId	IN	当前登录租客 ID
返回值	类型	OUT	说明
	类型		R<Contract>
	值		成功：code = 0，data 为新建合同（status = 0 待租客确认）
	值		失败：code = 2002 房源状态不允许签约；code = 1000 租期参数非法
详细说明			
读取房源与双方用户信息，渲染合同条款模板为 clauses JSON，按月租金与押金写入合同主表，同时生成风险条款集合 risk_flags 供解读标红；初始状态为待租客确认。			
使用注意事项			
合同条款以 JSON 数组存储，便于逐条解读与风险标注；生成合同不改变房源状态，须待双方签署生效后才置为已出租。			

函数名	signContract		
文件名	ContractController.java / ContractService.java		
功能概要	合同签署：租客签署后待房东确认，双方签署后合同生效并派生订单		
参数			
类型	变量名	I/O	说明
long	id	IN	合同 ID
ContractActionReq	req	IN	action（sign / reject / terminate，语义按当前用户角色判定）
long	uid	IN	当前登录用户 ID
int	role	IN	当前登录用户角色

函数名	signContract		
返回值	类型	OUT	说明
	类型		R<Map>
	值		成功：code = 0，返回合同当前状态及派生订单编号（生效时）
	值		失败：code = 3003 合同当前状态不允许该操作；code = 1007 无权限执行该操作
详细说明			
租客 sign 后合同流转为待房东确认并记录签署时间；房东 sign 后合同生效，在同一事务内派生租赁订单（contract_id 唯一）、按月生成租金账单计划、将房源状态置为已出租，并通知双方。账单生成以 UNIQUE(lease_order_id, period_no) 保证幂等。			
使用注意事项			
合同生效是整条交易链路的汇聚点，四个动作必须同事务完成，任一失败整体回滚；terminate 仅对已生效合同开放，终止后房源恢复可上架状态。			

函数名	generateBills		
文件名	ContractService.java		
功能概要	按租期逐月生成租金账单计划，重复执行保持幂等		
参数			
类型	变量名	I/O	说明
LeaseOrder	order	IN	已生效的租赁订单（含租期、月租与押金）
localDate	start	IN	租期起始日（账单分期基准）
返回值	类型	OUT	说明
	类型		List<RentBill>
	值		成功：返回按 period_no 升序排列的账单列表（含到期日与金额）
	值		失败：抛出业务异常，由事务整体回滚，不产生部分账单
详细说明			
以租期起止计算账期数，逐期生成 rent_bill 记录（period_no 自 1 递增、due_date 按自然月推算、amount 取月租，末期可按实际天数折算）；插入依赖 UNIQUE(lease_order_id, period_no)，重复调用不产生重复账单。			
使用注意事项			
账单为记录性质：系统不对接真实支付渠道，payBill 仅将状态置为已支付并记录支付时间；金额使用 DECIMAL(10,2)，禁止浮点参与计算以避免精度误差。			

5.2.6 评价与消息模块

评价与消息模块共包含 3 个对外接口函数，规约如下。

函数名	createReview
文件名	ReviewController.java / ReviewService.java
功能概要	提交评价：仅已完成合同可评、一单一评，并刷新房源均分

函数名	createReview		
参数			
类型	**变量名**	**I/O**	**说明**
ReviewReq	req	IN	leaseOrderId 租赁订单 ID、houseScore 房源分（1~5）、landlordScore 房东分（1~5）、content 文字内容
long	tenantId	IN	当前登录租客 ID
返回值	**类型**	OUT	**说明**
	类型		R<Review>
	值		成功：code = 0，data 为新建评价
	值		失败：code = 3005 仅完成合同后可评价且一单一评；code = 1007 非本人订单
详细说明			
校验订单存在、属于当前租客且状态为已完成，并确认该订单尚无评价；写入 review 后，在同一事务内重算并刷新 house.avg_score 冗余均分。			
使用注意事项			
唯一约束 UNIQUE(lease_order_id) 是“一单一评”的最终保障，业务层校验用于给出友好提示；冗余均分字段避免列表页对评价表做高频聚合（NFR-01）。			

函数名	listHouseReviews		
文件名	ReviewController.java / ReviewService.java		
功能概要	按房源分页查询评价列表		
参数			
类型	**变量名**	**I/O**	**说明**
long	houseId	IN	房源 ID
long	page	IN	页码，默认 1
long	size	IN	每页条数，默认 10
返回值	**类型**	OUT	**说明**
	类型		R<PageVO<TradeDto.ReviewVO>>
	值		成功：code = 0，data = { list, total, page, size }，每项含评分、内容、评价人与时间
	值		失败：code = 1000 分页参数非法
详细说明			
按房屋 ID 分页查询评价记录，过滤状态为已隐藏的评价，组装评价人昵称与头像后返回；列表按创建时间倒序排列。			
使用注意事项			
被管理员处理为违规的评价不对外展示，但记录保留以备追溯（不做物理删除）。			

函数名	sendNotification
文件名	NotificationService.java
功能概要	写入站内通知：全系统关键业务事件的统一消息出口

函数名	sendNotification		
参数			
类型	**变量名**	**I/O**	**说明**
Long	userId	IN	接收人用户 ID
int	type	IN	通知类型：1 审核 / 2 预约 / 3 签约 / 4 账单 / 5 举报处理
String	title	IN	通知标题
String	content	IN	通知正文
String	refType	IN	关联业务类型（house / appointment / contract …），可为空
Long	refId	IN	关联业务主键，可为空
返回值	**类型**	OUT	**说明**
	类型		void
	值		成功：通知记录写入 notification 表，接收方未读数加一
	值		失败：写入异常向上抛出，由调用方事务决定是否回滚；通知失败不影响主业务提交语义
详细说明			
向 notification 表插入一条未读通知，携带业务关联字段供前端点击跳转；由房源审核、预约状态变更、合同生效、账单生成与举报处理等多处调用，是模块间共享的服务而非某一模块私有实现。			
使用注意事项			
通知为业务副作用，调用方需明确其事务边界：审核、签约等关键动作中通知失败应随主事务回滚，以保证“有状态变更必有通知”；前端以轮询方式在 5 秒内可见（FR-21）。			

5.2.7 后台管理模块

后台管理模块共包含 6 个对外接口函数，规约如下。

函数名	pageUsers		
文件名	AdminController.java / AdminService.java		
功能概要	管理员分页查询用户列表，支持关键词检索		
参数			
类型	**变量名**	**I/O**	**说明**
String	keyword	IN	关键词（匹配用户名 / 昵称 / 手机号），可为空
long	page	IN	页码，默认 1
long	size	IN	每页条数，默认 10
返回值	**类型**	OUT	**说明**
	类型		R<PageVO<AdminDto.UserVO>>
	值		成功：code = 0，data = { list, total, page, size }，清单只含展示字段
	值		失败：code = 1007 无权限执行该操作（非管理员）

函数名	pageUsers
详细说明	
按关键词对用户名、昵称与手机号做模糊匹配，结果按注册时间倒序分页返回；响应中的密码字段不返回，手机号按需掩码。	
使用注意事项	
接口受 @RequireRole(3) 保护，仅管理员可访问；返回用户数据时须剔除密码等敏感字段，避免越权读取。	

函数名	setUserStatus		
文件名	AdminController.java / AdminService.java		
功能概要	禁用或启用用户账号，禁用即时生效		
参数			
类型	**变量名**	**I/O**	**说明**
long	id	IN	目标用户 ID
boolean	enabled	IN	true 启用，false 禁用
long	adminId	IN	当前管理员 ID（用于留痕）
返回值	**类型**	OUT	**说明**
	类型		R<Void>
	值		成功：code = 0，用户状态更新
	值		失败：code = 1006 用户不存在；code = 1007 无权限执行该操作
详细说明			
更新 sys_user.status 字段；由于 JwtAuthFilter 在每次请求时校验用户状态，被禁用用户的令牌虽未过期也会立即失去访问能力。操作结果写入审计日志。			
使用注意事项			
禁用是即时生效的强约束，设计上不依赖令牌过期或黑名单，避免维护额外状态；管理员不得禁用自身账号，防止平台失去管理入口。			

函数名	dashboard		
文件名	AdminController.java / AdminService.java		
功能概要	数据统计看板：核心指标总量与按日 / 周 / 月分组的趋势		
参数			
类型	**变量名**	**I/O**	**说明**
String	granularity	IN	统计粒度：day / week / month，默认 day
返回值	**类型**	OUT	**说明**
	类型		R<AdminDto.DashboardVO>
	值		成功：code = 0，data 含用户 / 房源 / 订单 / 预约的总量与趋势序列（含时间刻度与计数值）
	值		失败：code = 1000 granularity 取值非法；code = 1007 无权限执行该操作
详细说明			
对用户、房源、订单、预约等核心表执行分组聚合查询，按所选粒度对创建时间分组计数，组装为前端图表可直接消费的序列结构。			

函数名	dashboard
使用注意事项	
看板为只读聚合查询，不写入任何业务表，也不引入独立的统计汇总表，避免冗余数据与业务表不一致；数据量增长后可再考虑物化汇总。	

函数名	handleReport		
文件名	AdminController.java / ReportService.java		
功能概要	处理举报：记录处理意见并按目标类型执行差异化处置		
参数			
类型	**变量名**	**I/O**	**说明**
long	id	IN	举报 ID
String	remark	IN	处理说明
long	adminId	IN	当前管理员 ID
返回值	**类型**	OUT	**说明**
	类型		R<Void>
	值		成功：code = 0，举报状态置为已处理
	值		失败：code = 1007 无权限执行该操作
详细说明			
校验举报存在且处于待处理状态，写入处理人与处理说明并置为已处理；随后按目标类型差异化处理：评价类举报直接隐藏被举报评价，房源类举报向房东推送整改通知。处理动作写入审计日志。			
使用注意事项			
举报处理必须留痕且可追溯；评价处置采用隐藏而非删除，保留原始记录以备复核；房源类举报不做自动下架，由管理员结合虚假房源检测结果人工裁决。			

函数名	chats		
文件名	AdminController.java / AdminChatService.java		
功能概要	AI 对话审计列表：全站会话分页查询（含归属用户与对话规模指标）		
参数			
类型	**变量名**	**I/O**	**说明**
String	keyword	IN	关键词（匹配会话标题、用户昵称 / 账号 / 手机号），可为空
Integer	scene	IN	会话场景 1 找房助手 / 2 智能客服 / 3 合同解读，可为空表示全部
Boolean	transferred	IN	是否只看已转人工会话，可为空表示全部
Long	userId	IN	按归属用户过滤，可为空表示全部用户
long	page	IN	页码，默认 1
long	size	IN	每页条数，默认 10
返回值	**类型**	OUT	**说明**
	类型		R<PageVO<AdminChatDto.SessionVO>>
	值		成功：code = 0，data = { list, total, page, size }；每行含会话（场景 / 标题 / 转人工）、归属用户（昵称 / 账号 / 手机号 / 角色）、规模（轮次 / 消息数 / 工具调用数 / token）与末条消息预览

函数名	chats		
	值		失败：code = 1007 无权限执行该操作（非管理员）
详细说明			
按更新时间倒序分页返回全站会话，不限归属用户。会话规模不做逐会话查询：先对命中的 session_id 集合执行一次 GROUP BY session_id 聚合（消息数、用户轮次、工具调用数、token 合计、末条消息 id），再批量取出末条消息生成预览，避免 N+1 查询。关键词先匹配用户表，命中则以用户 id 集合与会话标题做 OR 匹配。			
使用注意事项			
只读接口，不提供修改与删除入口——对话日志只增不改，保证审计链路完整（NFR-05）。该接口是 FR-24“AI 对话量”指标的明细入口，汇总值由看板接口提供。			

函数名	chatDetail		
文件名	AdminController.java / AdminChatService.java		
功能概要	AI 会话轨迹：单会话逐步消息（含工具入参 / 返回）、引用来源、规模统计与角色设定原文		
参数			
类型	**变量名**	**I/O**	**说明**
long	id	IN	会话 ID（路径参数）
返回值	**类型**	OUT	**说明**
	类型		R<AdminChatDto.DetailVO>
	值		成功：code = 0，data = { session, currentEngine, hasRuleEngineTurn, systemPrompt, messages[], stats }
	值		失败：code = 1007 无权限执行该操作；code = 4002 会话不存在
详细说明			
按主键升序返回会话全部消息：role = 1 用户、2 助手、3 工具调用。工具消息的 tool_args 与 tool_result 在库内为 JSON 列，合法时解析为对象返回，供前端结构化展示；解析失败原样返回字符串。stats 给出轮次、助手消息数、工具调用数、token 合计、平均回答时延（只统计助手消息，工具耗时另在工具明细中给出）以及按工具名聚合的调用次数与平均耗时。			
使用注意事项			
会话历史未逐条快照所用模型，故 currentEngine 表示查看时刻生效的引擎；系统同一时刻只有一个生效模型系统同一时刻只有一个生效模型（配置驱动），跨模型的会话需结合配置变更时间人工判读；如日后需要逐条快照，需给 ai_chat_message 增加 model 列。			

5.3 关键数据结构定义

本节定义跨模块共享的关键数据结构。所有对外接口的响应统一以 R<T> 包装；分页查询统一返回 PageVO<T>；入参以各模块的 DTO 记录（record）承载并配合 Bean Validation 注解做参数校验；出参中需要跨表拼装的场景同样以专用 VO 记录承载（不再使用 Map，以便 OpenAPI 生成 schema、前端类型可对齐）。JSON 列的读写统一由 JsonColumns 承担，避免同一套“序列化 + 容错解析”在多个服务里各写一份。以下按统一响应、分页结构、核心 DTO、核心 VO 四类列出。

5.3.1 统一响应结构

```
/** 统一响应体：code = 0 成功；非 0 按 1xxx 用户 / 2xxx 房源 / 3xxx 交易 / 4xxx AI / 5xxx 系统分段 */
public class R<T> {
    private int    code;      // 业务状态码
    private String message;   // 提示信息（中文，可直接展示）
    private T      data;      // 业务数据，可为 null

    public static <T> R<T> ok(T data) { ... }
    public static R<Void>  ok()       { ... }
    public static <T> R<T> err(int code, String message) { ... }
}
```

5.3.2 分页结构

```
/** 分页响应包装：page 从 1 开始，size 默认 10、上限 50 */
public record PageVO<T>(List<T> list, long total, long page, long size) {
    public static <T> PageVO<T> of(IPage<T> p) {
        return new PageVO<>(p.getRecords(), p.getTotal(), p.getCurrent(), p.getSize());
    }
    /** 实体页 → VO 页：保留分页元数据，省掉各处“重建 Page”的样板 */
    public static <E, V> PageVO<V> map(IPage<E> page, Function<E, V> mapper) {
        return new PageVO<>(page.getRecords().stream().map(mapper).toList(),
                page.getTotal(), page.getCurrent(), page.getSize());
    }
}
```

5.3.3 核心 DTO 结构

入参 DTO 统一以 record 定义并配合 Bean Validation 注解完成参数校验，校验失败由全局异常处理器转换为 1000 错误码。

```
// ── 用户与权限（AuthDto）──────────────────────────────────────
public record CaptchaReq(@NotBlank @Pattern(regexp = "\\d{11}") String phone) {}
public record RegisterReq(@NotBlank String phone, @NotBlank String captcha,
                          @NotBlank @Size(min = 6, max = 32) String password,
                          @NotNull Integer role,          // 1 租客 / 2 房东
                          String nickname) {}
public record LoginReq(@NotBlank String username, @NotBlank String password) {}
public record UpdateProfileReq(String nickname, String email, String avatarUrl) {}
```

```
    public record ChangePasswordReq(@NotBlank String oldPassword,
                                    @NotBlank @Size(min = 6, max = 32) String newPassword) {}
    public record RealnameReq(@NotBlank String realName,
                              @NotBlank @Pattern(regexp = "\\d{17}[\\dXx]") String idCardNo) {}

    // ── 房源（HouseDto）──────────────────────────────────────────
    public record SaveReq(@NotBlank String title, @NotBlank String community,
                          @NotBlank String city, @NotBlank String district,
                          @NotBlank String address, @NotBlank String layout,
                          @NotNull @DecimalMin("1") BigDecimal area,
                          String orientation, String floorDesc,
                          @NotNull @DecimalMin("1") @DecimalMax("1000000") BigDecimal rent,
                          @NotBlank String depositType, List<String> facilities,
                          String description,
                          @NotNull @DecimalMin("-180") @DecimalMax("180") BigDecimal lng,
                          @NotNull @DecimalMin("-90")  @DecimalMax("90")  BigDecimal lat,
                          List<String> images) {}
    public record StatusReq(@NotBlank String action) {}   // 上架 / 下架
    public record SearchReq(String keyword, String district, String layout, String orientation,
                            BigDecimal rentMin, BigDecimal rentMax, List<String> facilities,
                            BigDecimal lngMin, BigDecimal lngMax,
                            BigDecimal latMin, BigDecimal latMax,
                            String sort,                 // rent_asc / rent_desc / hot / new
                            Long page, Long size) {}
    public record AiFillReq(String title, String community, String layout, String
    imageFileName) {}

    // ── 交易（TradeDto）──────────────────────────────────────────
    public record AppointmentCreateReq(@NotNull Long houseId,
                                       @NotNull LocalDateTime appointmentTime, String remark)
    {}
    public record AppointmentActionReq(@NotBlank String action, String reason) {}
                            // confirm / reject / cancel / complete
    public record ContractCreateReq(@NotNull Long houseId,
                                    @NotNull LocalDate startDate, @NotNull LocalDate endDate)
    {}
    public record ContractActionReq(@NotBlank String action) {}  // sign / reject / terminate
    public record ReviewReq(@NotNull Long leaseOrderId,
                            @NotNull @Min(1) @Max(5) Integer houseScore,
                            @NotNull @Min(1) @Max(5) Integer landlordScore, String content) {}
    public record ReportReq(@NotNull Integer targetType, @NotNull Long targetId,
                            @NotBlank String reason) {}

    // ── AI 智能体（AiDto）────────────────────────────────────────
    public record SessionCreateReq(@NotNull @Min(1) @Max(3) Integer scene, String title) {}
                            // 1 找房助手 / 2 智能客服 / 3 合同解读
```

```
public record MessageSendReq(@NotNull String content) {}
```

5.3.4 核心 VO 结构

出参中需要跨表拼装或承载 AI 分析结果的对象以 VO 定义，如下所示。

```
// ── 认证与用户 ──────────────────────────────────────────────
public record TokenResp(String token, Long userId, Integer role, String nickname) {}
public record RealnameVO(String realName, Integer status,
                         String rejectReason, String maskedIdCard) {}

// ── 房源 ──────────────────────────────────────────────────
public record Item(House house, List<Image> images, String landlordName,
                   Boolean favorited, Long reviewCount) {
    public record Image(Long id, String url, Integer sort, Boolean isCover) {}
}
public record AiFillVO(String description, String orientation,
                       String floorDesc, List<String> facilities) {}

// ── AI 流式事件与三类分析结果 ──────────────────────────────
public record StreamEvent(String type, String delta, Long messageId,
                          List<Map<String, String>> citations, String suggestion) {}
                         // type: delta 逐字片段 / done 收尾（含引用与转人工建议）
public record PricingVO(BigDecimal low, BigDecimal high, BigDecimal avg,
                        String basis, Integer sampleCount, String note, String model) {}
public record DetectVO(Integer riskScore, List<String> suspicions,
                       String suggestion, String model) {}
public record InterpItem(Integer index, String clause, Boolean risk, String explanation) {}
public record InterpVO(List<InterpItem> items, String disclaimer, String model) {}

// ── 后台 AI 对话审计（AdminChatDto，仅管理员可见）──────────
// SessionVO：会话 + 人物（归属用户与角色）+ 规模（轮次/消息数/工具调用数/token）
// DetailVO(currentEngine, hasRuleEngineTurn, systemPrompt, messages, stats)
// MessageVO：role 1 用户 / 2 助手 / 3 工具调用，工具消息带 toolName / toolArgs / toolResult
// StatsVO：roundCount, assistantCount, toolCallCount, totalTokens, avgLatencyMs, tools[]
public record SessionVO(Long id, Integer scene, String sceneName, String title,
                        Long userId, String nickname, String username, String phone,
                        Integer userRole, Integer isTransferred, Integer messageCount,
                        Integer roundCount, Integer toolCallCount, Integer totalTokens,
                        String lastMessage, LocalDateTime createdAt,
                        LocalDateTime updatedAt) {}
public record MessageVO(Long id, Integer role, String roleName, String content,
                        String toolName, Object toolArgs, Object toolResult,
                        List<Map<String, Object>> citations, Integer tokenCount,
                        Integer latencyMs, LocalDateTime createdAt) {}
```